

트래블버드 통합 백엔드 기능명세서 결정사항 반영본 v5

최종 결정사항 통합본 v5

기준일: 2026-08-06

문서 적용 원칙

- API 기본 Prefix는 `/api/` 로 한다.
- 서버 내부 시간은 UTC로 저장하고 클라이언트에는 ISO 8601 형식으로 반환한다.
- 날짜 기준 상태 판정과 일일 제한은 대한민국 서비스 기준으로 `Asia/Seoul` 날짜를 사용한다.
- MySQL 문자셋은 `utf8mb4` 를 사용한다.
- JPA Entity는 기본적으로 지연 로딩을 사용하며 N 관계는 연결 Entity로 분리한다.
- API 응답에서 JPA Entity를 직접 반환하지 않고 Request-Response DTO를 사용한다.
- 모든 사용자 소유 데이터는 JWT의 사용자 ID를 기준으로 권한을 검증하며 Request Body의 사용자 ID를 신뢰하지 않는다.
- 비로그인 접근이 허용된 API는 선택적 인증을 적용하고, 로그인 필수 API는 인증정보가 없으면 `401 UNAUTHORIZED` 를 반환한다.
- API URI, DTO명, Entity명, DB 컬럼명과 타입은 Spring Boot·JPA·MySQL 환경을 기준으로 작성한 구현 초안이다.

1. 서비스 개요

1.1 한 줄 서비스 정의

- 트래블버드는 대한민국 국내 여행에 특화된 여행 일정 계획·기록·공유 서비스다.
- 사용자는 직접 또는 AI의 도움을 받아 여행 일정을 만들고, 여행 중 사진과 메모를 남기며, 여행 후 기록을 완성하여 다른 사용자와 공유할 수 있다.

1.2 서비스 목적

- 여행 전에는 직접 일정 또는 AI 추천 일정으로 여행을 계획한다.
- 여행 중에는 일정에 연결된 장소별로 사진과 한줄 메모를 추가한다.
- 여행 후에는 일정과 장소 정보를 기반으로 여행 기록을 완성한다.
- 여행 준비부터 기록까지 하나의 서비스 흐름으로 연결한다.
- 공개된 기록은 커뮤니티에서 다른 사용자에게 공유된다.
- 다른 사용자는 공개 기록을 통해 새로운 장소와 여행 경로를 발견할 수 있다.
- 사용자가 자신의 여행을 사진과 글로 감성적으로 기록하고 보관할 수 있도록 한다.
- 사용자가 방문한 장소와 지역을 포토맵에 축적하여 여행 성취감을 느낄 수 있고 시각적으로 확인한다.
- 온보딩 성향 테스트를 통해 5종의 파트너 새 중 하나를 배정하여 서비스 정체성을 제공한다.
- 캐릭터 경험치·레벨·성장 단계와 리워드 포인트 기능은 MVP에서 제공하지 않는다.
- 서비스는 전 세계 여행 편의 기능보다 국내 여행 기록·공유의 감성적 경험에 집중한다.

1.3 핵심 서비스 도메인

- 인증·회원 도메인은 카카오 로그인, 로그아웃, 회원 탈퇴와 프로필 관리를 담당한다.
- 온보딩·파트너 새 도메인은 성향 테스트와 파트너 새 종류 배정을 담당한다.
- 장소 도메인은 국내 장소 검색, 상세 조회와 장소 저장을 담당한다.
- 일정 도메인은 직접 일정, Day별 장소, 위시리스트와 여행 캘린더를 담당한다.
- 기록 도메인은 여행 기록 작성, 공개 범위, 이미지와 장소 연결을 담당한다.
- 커뮤니티 도메인은 공개 기록 탐색, 검색, 인기 필터와 이웃새 게시글을 담당한다.
- 저장 도메인은 장소 저장과 다른 사용자의 여행 경로 저장을 담당한다.
- 포토맵 도메인은 게시글과 연결된 방문 장소를 지역별로 집계한다.
- 홈 도메인은 추천 장소, 축제·이벤트와 추천 기록을 제공한다.
- AI 도메인은 여행 조건, 저장 장소 또는 위시리스트를 기반으로 Day별 추천 일정을 생성한다.
- 운영 안전 도메인은 게시글 신고, 게시글 차단 상태와 사용자 차단을 담당한다.

- 관리자 웹 페이지, 관리자 전용 API, 캐릭터 성장과 리워드 포인트 도메인은 MVP 범위에 포함하지 않는다.

1.4 기술 아키텍처 요약

- 모바일 클라이언트는 React Native로 구현한다.
- 백엔드는 Spring Boot와 Spring Security를 사용한다.
- 데이터베이스는 MySQL을 사용한다.
- 인증은 JWT Access Token과 Refresh Token을 사용한다.
- 외부 지도와 장소 검색 기능은 네이버 API를 사용한다.
- 인프라는 AWS EC2, RDS, S3 등을 사용한다.
- 권장 요청 흐름은 `React Native → HTTPS/ALB → Spring Boot → RDS·S3·네이버 API·AI 서버` 다.
- 초기 서비스는 모듈형 모놀리스 구조로 한다.
- 이미지 원본은 S3에 저장하고 DB에는 파일 메타데이터와 S3 Object Key만 저장한다.
- 운영 DB는 AWS RDS MySQL에 배치한다.
- JWT Secret, 네이버 API Key와 DB 비밀번호는 AWS Secrets Manager 또는 Parameter Store에서 관리한다.
- Refresh Token은 Redis를 사용하지 않고 MySQL DB에서만 관리한다.
- 백엔드는 AI 서버에 HTTP REST로 추천 작업을 전달한 뒤 생성 결과를 기다리지 않고 연결을 종료한다. AI 서버는 처리가 끝나면 백엔드 내부 Callback API로 결과를 전달하며, 클라이언트는 `jobId` 를 이용한 폴링 방식으로 상태를 확인한다.
- 게시글이 최초 발행된 Trip은 해당 Post가 삭제될 때까지 경로 구조를 잠그며, SavedRoute는 잠긴 원본 경로를 참조한다.
- 이미지 원본은 서버에 보관하지 않고 프론트엔드가 최적화한 WebP 파일만 S3에 업로드한다.
- 카카오 로그인 입력은 React Native 카카오 SDK가 발급한 카카오 Access Token 한 가지 방식으로 통일한다.

2. 주요 이용자 및 권한

2.1 비로그인 사용자

- 비로그인 사용자는 커뮤니티의 `ALL`, `POPULAR` 게시글 목록을 조회할 수 있다.
- 비로그인 사용자는 `PUBLIC`, `MEMO_PRIVATE` 게시글 상세를 조회할 수 있다.
- 비로그인 사용자는 `PUBLIC`, `MEMO_PRIVATE` 일정 상세를 공유 URL로 조회할 수 있다.
- 비로그인 사용자가 장소 검색, 장소 상세 조회, 장소 저장, 팔로우, 사용자 차단, 일정 생성·수정, AI 추천 등 로그인 필수 기능을 요청하면 `401 UNAUTHORIZED` 를 반환한다.
- 비로그인 사용자의 게시글 조회는 조회수에 집계하지 않는다.
- 비공개 게시글과 비공개 일정은 비로그인 사용자에게 존재 여부를 노출하지 않고 `404` 로 처리한다.

2.2 로그인 사용자

- 로그인 사용자는 성향 테스트와 파트너 새 배정을 이용할 수 있다.
- 로그인 사용자는 프로필 조회와 수정을 이용할 수 있다.
- 로그인 사용자는 장소를 검색·조회하고 저장하거나 저장 취소할 수 있다.
- 로그인 사용자는 직접 일정을 생성하고 Day별로 장소를 추가·삭제·정렬할 수 있다.
- 로그인 사용자는 장소 위시리스트를 만들 수 있다.
- 로그인 사용자는 여행 기록을 임시 저장하거나 발행하고 수정·삭제할 수 있다.
- 로그인 사용자는 자신의 일정과 게시글을 `PUBLIC`, `MEMO_PRIVATE`, `PRIVATE` 로 설정할 수 있다.
- 로그인 사용자는 다른 사용자의 공개 경로를 저장할 수 있다.
- 로그인 사용자는 다른 사용자를 이웃새로 팔로우하거나 차단할 수 있다.
- 로그인 사용자는 자신의 포토맵, 통계, 일정과 캘린더를 조회할 수 있다.
- 로그인 사용자는 AI 일정 추천을 하루 최대 3회 요청할 수 있다.
- 사용자는 자신의 여행 일정, 게시글, 일정 내 장소와 사진만 수정·삭제할 수 있다.
- 발행된 Post가 연결된 Trip은 경로 구조를 수정할 수 없고, 제목·본문·사진·메모·동행 유형·테마·설명·추천 시간·체류시간 등 경로와 무관한 정보만 수정할 수 있다.
- Spring Security URL 인가 외에도 Service 계층에서 리소스 소유권을 검증한다.

2.3 관리자 등 기타 사용자

- `users.role` 은 `ROLE_USER`, `ROLE_ADMIN` 으로 구분한다.
- 신규 가입자의 기본 권한은 `ROLE_USER` 다.

- 운영 관리자 계정은 DB에서 `ROLE_ADMIN` 으로 지정할 수 있다.
- 관리자 웹 페이지와 `/api/admin/**` 전용 API는 구현하지 않는다.
- 사용자용 게시글 신고 API `POST /api/reports` 만 최소 기능으로 제공한다.
- 신고 내역은 `reports` 테이블에 저장한다.
- 신고 접수만으로 게시글을 자동 차단하지 않는다.
- 운영자는 DB에서 대상 게시글의 `status` 를 `BLOCKED` 로 변경하는 SQL을 직접 실행한다.
- 일반 게시글 목록·상세·검색·추천에서는 `status=BLOCKED` 인 게시글을 제외한다.

2.4 Spring Security 인증 정책

- JWT 인증은 `OncePerRequestFilter` 를 상속한 JWT Filter에서 처리한다.
- Filter는 `Authorization: Bearer {accessToken}` 헤더를 검증한다.
- JWT Payload에는 `userId` , `role` , `tokenType` , `iat` , `exp` 를 포함한다.
- Access Token은 Stateless하게 검증한다.
- Access Token 만료기간은 30분이다.
- Refresh Token 만료기간은 1개월이다.
- Refresh Token은 MySQL의 `refresh_tokens` 테이블에 해시값으로 저장한다.
- 사용자당 활성 Refresh Token은 하나만 허용한다.
- 활성 Refresh Token이 이미 존재하면 새로운 Access Token과 Refresh Token을 발급하지 않고 신규 로그인을 차단한다.
- `deviceId` 는 사용하지 않는다.
- 로그아웃 시 Refresh Token만 폐기한다.
- Access Token 즉시 무효화를 위한 Redis 블랙리스트는 구현하지 않는다.
- 로그아웃 이후 기존 Access Token은 최대 30분 동안 유효할 수 있으며 만료시간이 지나면 자연 만료된다.
- 인증 오류는 `AuthenticationEntryPoint` , 권한 오류는 `AccessDeniedHandler` 에서 공통 JSON으로 반환한다.

3. 백엔드 상세 기능 명세

3.1 인증 및 회원

3.1.1 기능명: 카카오 회원가입 및 로그인

기능 설명

- 사용자는 카카오 계정을 통해 로그인한다.
- 카카오 최초 로그인 사용자는 신규 회원으로 저장한다.
- 로그인 성공 시 서비스 Access Token과 Refresh Token을 발급한다.
- 카카오 인증 입력 방식은 React Native 카카오 SDK가 획득한 `kakaoAccessToken` 을 Spring Boot가 검증하는 방식으로 통일한다.
- 카카오 인가 코드와 `redirectUri` 를 백엔드 로그인 요청으로 전달하는 방식은 사용하지 않는다.
- 카카오 이메일은 선택 동의로 수집하며 제공되지 않아도 로그인을 허용한다.
- 카카오 프로필 이미지는 수집·저장하지 않는다.
- 서비스 프로필은 온보딩 테스트 후 지정되는 5종의 파트너 새 아이콘을 사용한다.

세부 로직 및 상태 변화

- 클라이언트는 React Native 카카오 SDK로 로그인한 뒤 발급받은 카카오 Access Token을 백엔드에 전달한다.
- 서버는 전달받은 카카오 Access Token으로 카카오 사용자 정보 API를 호출하여 토큰 유효성과 카카오 사용자 고유 ID를 확인한다.
- 카카오 Access Token 원문은 인증 확인에만 사용하고 서버 DB와 로그에 저장하지 않는다.
- 카카오 사용자 고유 ID로 `user_social_accounts` 를 조회한다.
- 계정이 없으면 `users` 와 `user_social_accounts` 를 하나의 트랜잭션에서 생성한다.
- 신규 사용자는 `status=ACTIVE` , `role=ROLE_USER` , `onboardingCompleted=false` , `birdType=null` 로 저장한다.
- 카카오에서 이메일이 제공되면 저장하고, 제공되지 않으면 `users.email=null` 로 저장한다.
- 가짜 이메일을 생성하지 않는다.
- 기존 사용자는 사용자 상태가 `ACTIVE` 인지 검증한다.

- 해당 사용자에게 활성 Refresh Token이 있으면 신규 로그인을 차단한다.
- 활성 Refresh Token이 없을 때만 서비스 Access Token과 Refresh Token을 발급한다.
- Refresh Token 원문은 반환 후 서버에 보관하지 않고 해시값만 저장한다.
- 온보딩 완료 여부를 응답에 포함한다.

입력 / Request DTO

- DTO명은 `KakaoLoginRequest` 다.
- `kakaoAccessToken: String`
- `authorizationCode`, `redirectUri`, `deviceId` 는 Request DTO에 포함하지 않는다.

출력 / Response DTO

- `accessToken: String`
- `refreshToken: String`
- `accessTokenExpiresIn: Long` (1800)
- `refreshTokenExpiresIn: Long` (2592000 기준)
- `isNewUser: Boolean`
- `onboardingCompleted: Boolean`
- `userId: Long`
- `nickname: String?`
- `birdType: BirdType?`

예외사항 및 검증 로직

- `kakaoAccessToken` 이 누락되면 `400 INVALID_AUTH_REQUEST` 를 반환한다.
- 카카오 토큰 검증에 실패하면 `401 KAKAO_AUTHENTICATION_FAILED` 를 반환한다.
- 정지 계정이면 `403 USER_NOT_ACTIVE` 를 반환한다.
- 활성 Refresh Token이 존재하면 `409 ACTIVE_SESSION_ALREADY_EXISTS` 를 반환한다.
- 카카오 이메일 미제공은 오류로 처리하지 않는다.
- `provider + providerUserId` 유니크 제약으로 중복 계정 생성을 방지한다.

- 카카오 외부 API 장애 시 `503 KAKAO_SERVICE_UNAVAILABLE` 을 반환한다.

연동 API 엔드포인트

- `POST /api/auth/kakao/login`

3.1.2 기능명: JWT 토큰 재발급

기능 설명

- 만료된 Access Token을 유효한 Refresh Token으로 재발급한다.
- 사용자당 하나의 활성 Refresh Token만 유지한다.

세부 로직 및 상태 변화

- Refresh Token의 서명과 만료시간을 검증한다.
- Refresh Token에서 사용자 ID를 확인한다.
- MySQL DB에 저장된 해당 사용자의 Refresh Token 해시와 비교한다.
- 사용자 상태가 `ACTIVE` 인지 확인한다.
- 검증에 성공하면 기존 Refresh Token을 폐기하고 새로운 Access Token과 Refresh Token으로 교체한다.
- 재사용되거나 DB 값과 불일치하는 토큰은 거부한다.
- Redis와 `deviceId` 는 사용하지 않는다.

입력 / Request DTO

- `refreshToken: String`

출력 / Response DTO

- `accessToken: String`
- `refreshToken: String`
- `accessTokenExpiresIn: Long`
- `refreshTokenExpiresIn: Long`

예외사항 및 검증 로직

- 만료된 Refresh Token은 `401 REFRESH_TOKEN_EXPIRED` 다.
- 폐기된 토큰은 `401 REFRESH_TOKEN_REVOKED` 다.
- DB 저장값과 일치하지 않으면 `401 INVALID_REFRESH_TOKEN` 이다.
- 정지·삭제 사용자 토큰이면 `403 USER_NOT_ACTIVE` 다.

연동 API 엔드포인트

- `POST /api/auth/token/refresh`
-

3.1.3 기능명: 로그아웃

기능 설명

- 사용자는 현재 활성 로그인 세션에서 로그아웃한다.

세부 로직 및 상태 변화

- 전달받은 Refresh Token의 소유자와 JWT 사용자 ID가 일치하는지 검증한다.
- DB에서 Refresh Token 레코드를 삭제하거나 `revokedAt` 을 기록한다.
- 클라이언트는 기기에 저장된 Access Token과 Refresh Token을 삭제한다.
- Access Token 블랙리스트는 사용하지 않는다.
- Access Token은 발급 후 최대 30분 내 자연 만료된다.

입력 / Request DTO

- `refreshToken: String`

출력 / Response DTO

- 성공 시 `204 No Content`

예외사항 및 검증 로직

- 이미 폐기된 토큰으로 요청해도 멍등하게 `204 No Content` 를 반환한다.

- 타인의 Refresh Token이면 `403 TOKEN_ACCESS_DENIED` 를 반환한다.

연동 API 엔드포인트

- `POST /api/auth/logout`

3.1.4 기능명: 회원 탈퇴

기능 설명

- 사용자는 계정을 즉시 탈퇴할 수 있다.
- 탈퇴 후 동일 카카오 계정으로 재가입할 수 있다.

세부 로직 및 상태 변화

- 모든 Refresh Token을 폐기한다.
- 탈퇴 사용자의 공개·비공개 게시글을 삭제 처리한다.
- 게시글과 연결된 이미지 파일을 S3와 DB에서 삭제한다.
- 일정, Day, 일정 장소, 일정 장소 메모·사진, 위시리스트, 저장 장소, 포토맵 대상 데이터를 요청 시점에 삭제한다.
- 팔로우·차단 관계와 사용자 소유 신고 부가정보를 정리한다.
- `user_social_accounts` 의 카카오 연결정보를 제거하여 동일 카카오 계정 재가입을 허용한다.
- 게시글 원본을 참조한 타 사용자의 저장 관계는 삭제하지 않고 `sourceAvailable=false` 로 변경하여 원본 내용을 노출하지 않는다.
- 대량 S3 삭제는 탈퇴 API 트랜잭션 완료 후 실패 재처리가 가능한 삭제 작업 테이블로 처리할 수 있으나, 사용자 화면에서는 즉시 삭제된 것으로 간주한다.

입력 / Request DTO

- `reasonCode: String?`
- `confirmationText: String?`

출력 / Response DTO

- 성공 시 `204 No Content`

예외사항 및 검증 로직

- 이미 탈퇴한 요청은 역등하게 처리한다.
- Access Token 사용자와 탈퇴 대상은 일치해야 한다.

연동 API 엔드포인트

- `DELETE /api/users/me`
-

3.2 프로필

3.2.1 기능명: 내 정보 조회

기능 설명

- 로그인 사용자의 닉네임, 자기소개와 파트너 새 종류를 조회한다.

세부 로직 및 상태 변화

- JWT 사용자 ID로 `users` 를 조회한다.
- 프론트엔드는 `birdType` 과 앱 내부 5종 이미지 에셋을 매핑한다.
- 카카오 프로필 이미지 또는 사용자 업로드 프로필 이미지를 사용하지 않는다.

출력 / Response DTO

- `userId: Long`
- `nickname: String?`
- `introduction: String?`
- `onboardingCompleted: Boolean`
- `birdType: BirdType?`
- `role: UserRole`

예외사항 및 검증 로직

- 존재하지 않는 사용자는 `404 USER_NOT_FOUND` 다.
- 온보딩 전이면 `birdType=null` 을 반환한다.

연동 API 엔드포인트

- `GET /api/users/me`
-

3.2.2 기능명: 프로필 수정

기능 설명

- 사용자는 닉네임과 자기소개를 수정할 수 있다.

세부 로직 및 상태 변화

- 변경된 필드만 수정하는 PATCH 방식을 사용한다.
- 닉네임 중복은 허용한다.
- 닉네임은 최소 2자, 최대 10자다.
- 자기소개는 공백 포함 최대 100자다.
- 프로필 이미지 수정 기능은 제공하지 않는다.

입력 / Request DTO

- `nickname: String?`
- `introduction: String?`

출력 / Response DTO

- 수정된 프로필 전체를 반환한다.

예외사항 및 검증 로직

- 닉네임 또는 자기소개 길이가 기준을 벗어나면 `400 INVALID_PROFILE_VALUE` 를 반환한다.

연동 API 엔드포인트

- `PATCH /api/users/me/profile`
-

3.3 온보딩 및 파트너 새

3.3.1 기능명: 성향 테스트 문항 조회

기능 설명

- 성향 테스트는 총 8개 질문과 문항별 5개 선택지로 구성한다.
- 각 선택지는 **미식**, **휴식**, **사진**, **액티비티**, **문화** 중 하나의 성향 점수를 가진다.
- 테스트를 건너뛸 수 없으며 온보딩 완료 전 필수로 수행한다.
- 사용자가 선택지 위치 패턴을 추측하지 못하도록 API 호출마다 각 문항의 선택지 배열 순서를 무작위로 섞어 반환한다.

세부 로직 및 상태 변화

- 활성화된 최신 테스트 버전의 질문을 Q1부터 Q8까지 고정 순서로 반환한다.
- 질문 순서는 유지하고, 각 질문에 속한 5개 선택지만 독립적으로 Random Shuffle한다.
- 선택지의 점수 성향은 **optionId** 와 서버 내부 매핑값으로 판정하며, 응답 배열 위치나 표시 순서로 점수를 계산하지 않는다.
- 셔플 이후 클라이언트 표시용 **order** 를 1부터 다시 부여한다.
- 결과 점수, 내부 성향 코드와 파트너 새 매핑 기준은 클라이언트에 노출하지 않는다.
- 테스트 문항 변경에 대비해 질문 세트의 **testVersion** 을 관리한다.

캐릭터 성향 질문 리스트

1. Q1. 여행지에서 정체불명의 초대장을 받았다. 초대장에 적혀있으면 하는 문구는?

- **[미식]** 아무나 맛볼 수 없는, 오늘 단 하루만 열리는 미식의 정점에 당신을 초대합니다.
- **[휴식]** 오늘만큼은 서두르지 마세요. 당신만의 완벽한 쉼터로 초대합니다.
- **[사진]** 셔터를 누르는 순간 영화가 되는 곳, 당신만의 인생 한 장면을 선물해 드립니다.
- **[액티비티]** 심장이 터질 듯한 새로운 도파민의 세계가 곧 열립니다.
- **[문화]** 당신의 눈과 귀를 사로잡고 마음에 깊은 영감을 새겨줄 현장으로 초대합니다.

2. Q2. 여행지에 도착한 직후, 내 눈앞에 펼쳐졌으면 하는 장면은?

- **[액티비티]** 지루한 일상을 단번에 날려버릴 만큼 활기차고, 온몸의 세포를 깨워줄 짜릿한 모험과 즐길 거리가 가득한 핫플레이스
- **[사진]** 창밖으로 수채화처럼 붉게 물들어가는 노을과, 그 실루엣이 액자처럼 담기는 빈티지 카페의 창가
- **[문화]** 수십 년의 세월을 간직한 골목과 은은한 아날로그 감성이 묻어나는 오래된 문화 거리
- **[휴식]** 잔잔하게 들리는 파도 소리와 은은한 나무 향이 가득한 프라이빗하고 아늑한 숙소
- **[미식]** 침샘을 자극하는 맛있는 음식 냄새와, 달콤한 디저트 냄새가 뒤섞인 활기찬 거리

3. Q3. 숙소 근처를 걷다가 다섯 갈래 길을 발견했다. 어느 길로 가볼까?

- **[사진]** 스냅사진 속 한 장면처럼 초록 식물들이 하얀 벽을 감싸고 있는, 카메라를 켜 수밖에 없는 예쁜 주택가 길
- **[문화]** 수십 년의 세월을 간직한 오래된 건축물과 고즈넉한 돌담길을 따라 느긋하게 걷는 역사적인 길
- **[휴식]** 따뜻한 온기와 은은한 음악이 기다리는 숙소로 향하는 잔잔하고 편안한 길
- **[미식]** 코끝을 강렬하게 자극하는 맛있는 음식 냄새를 따라, 현지인들이 옹기종기 줄 서 있는 유명 먹거리 길
- **[액티비티]** 멀리서 웃음소리와 활기찬 에너지가 뿜어져 나오는, 재미있는 즐길 거리가 가득해 보이는 북적이는 길

4. Q4. 이번 여행에서 하루 동안 특별한 능력을 하나 얻는다면?

- **[휴식]** 시끄러운 인파 속에서도 머릿속이 비워지며 완벽한 이너피스를 찾는 능력
- **[액티비티]** 아침부터 새벽까지 쉬지 않고 돌아다녀도 방전되지 않는 무한 체력
- **[문화]** 오래된 돌담길이나 건물만 봐도 그곳에 얹힌 흥미진진한 이야기를 알아채는 능력
- **[미식]** 간판만 속 봐도 실패 확률 0%의 로컬 찐맛집을 단번에 감별하는 능력
- **[사진]** 평범한 골목길에서도 필터 낀 듯 감성적인 구도를 포착해 내는 스냅 작가의 눈

5. Q5. 이제 이 여행지에서 내게 주어진 시간은 단 3시간. 이 중 제일 포기 못하는 것은?

- **[사진]** 3시간이면 인생샷 300장은 건진다! 무조건 뷰가 예쁜 핫플레이스로 직진.

- **[미식]** 웨이팅이 길어도 상관없다. 이 지역에서 가장 유명한 로컬 맛집으로 달려간다.
- **[휴식]** 3시간뿐인데 쫓기듯 다니기 싫다... 여기저기 돌아다니지 않고, 편안한 공간에서 좋아하는 음악 들으며 쉬기.
- **[액티비티]** 지루하게 보낼 틈이 없어! 남은 시간을 꽉 채워줄 활기차고 역동적인 놀거리를 찾아 발 빠르게 움직인다.
- **[문화]** 이 지역의 진짜 정취를 채우고 싶어, 로컬 감성이 가득한 문화 공간이나 소품샵 투어하기.

6. Q6. 내가 꿈꾸는 '완벽한 여행의 마무리'는?

- **[휴식]** 따뜻한 물로 샤워를 마치고, 폭신한 침대 속으로 쏙 들어가, 밀린 피로를 풀며 꿀잠 자기
- **[액티비티]** 힙한 펍에서 밤새 다트 게임을 즐기거나, 야간 야외 수영장에서 수영하며 보내기
- **[문화]** 은은한 달빛이 내리는 골목과 문화재 야경을 느긋하게 산책하며, 이 도시의 밤을 온전히 느끼기
- **[미식]** 숙소 테이블에 지역 유명 야식과 시원한 맥주를 잔뜩 펼쳐놓고 맛있게 먹으며 수다 떨기
- **[사진]** 스탠드 불빛 아래에서 오늘 찍은 사진들을 짚 훑어보고, 마음에 드는 인생샷을 골라 SNS나 일기장에 기록하기

7. Q7. 여행을 마무리하고 탄 기차 안에서 하는 말은?

- **[휴식]** "몸도 마음도 제대로 비우고 가네. 다시 시작할 힘이 좀 나는 것 같아."
- **[액티비티]** "진짜 알차게 놀았다! 다음엔 더 다이نام릭하고 재미 넘치는 곳으로 가야지."
- **[문화]** "그때 봤던 작품과 도시의 풍경이 아직도 잊히지 않네."
- **[미식]** "그 맛을 잊을 수가 없다... 또 먹으러 가고 싶어!"
- **[사진]** "사진 보정하고 있는데 버릴 사진이 하나도 없네. 역시 남는 건 사진뿐이야."

8. Q8. 여행을 마치고 열어본 내 갤러리는?

- **[휴식]** 아늑한 숙소와 조용한 카페에서 온전히 쉬어갔던 편안한 순간들의 흔적
- **[액티비티]** 온몸을 던져 신나게 뛰어놀며 찍힌, 흔들렸지만 생동감 넘치는 역동적인 순간들

- **[문화]** 이 지역 고유의 매력이 묻어나는 웅장한 건축물과 독특한 예술 작품이 담긴 사진들
- **[미식]** 비주얼만 봐도 군침이 도는 푸짐한 로컬 음식과 디저트 사진들로 가득한 화면
- **[사진]** 카메라 셔터만 눌러도 그림이 되는 포토존에서 남긴 나만의 감성 가득한 인생샷들

문서의 **[미식]**, **[휴식]**, **[사진]**, **[액티비티]**, **[문화]** 표시는 백엔드 점수 매핑을 설명하기 위한 내부 표기이며 실제 사용자 응답 DTO의 선택지 텍스트에는 노출하지 않는다.

출력 / Response DTO

- `testVersion: String`
- `questions`
 - `questionId: Long`
 - `text: String`
 - `order: Int`
 - `options`
 - `optionId: Long`
 - `text: String`
 - `order: Int` — 호출 시 셔플된 표시 순서

예외사항 및 검증 로직

- 활성 테스트가 없으면 `503 PERSONALITY_TEST_UNAVAILABLE` 을 반환한다.
- 이미 테스트를 완료한 사용자는 문항을 다시 조회할 수 있으나 재제출은 허용하지 않는다.

연동 API 엔드포인트

- `GET /api/onboarding/personality-test`

3.3.2 기능명: 성향 테스트 제출 및 파트너 새 배정

기능 설명

- 사용자가 8개 문항의 답변을 제출하면 선택지별 성향 점수를 합산한다.
- 최고 점수가 하나이면 해당 성향에 연결된 파트너 새를 즉시 배정한다.
- 최고 점수 성향이 둘 이상이면 서버가 임의 우선순위를 적용하지 않고 사용자가 동점 성향 중 하나를 직접 선택하도록 2단계 Tie-breaker 흐름을 사용한다.
- 테스트 재응시와 건너뛰기를 허용하지 않는다.

세부 로직 및 상태 변화

- 테스트 버전과 질문·선택지 유효성을 검증한다.
- 8개 필수 질문에 정확히 한 번씩 답했는지 검증한다.
- 선택지 표시 순서가 아니라 `optionId`에 연결된 내부 성향 코드로 점수를 합산한다.
- 내부 성향 코드는 `GOURMET`, `REST`, `PHOTO`, `ACTIVITY`, `CULTURE`다.
- 단독 최고 점수인 경우 다음을 하나의 트랜잭션으로 처리한다.
 1. 답변과 점수 결과를 저장한다.
 2. 성향에 연결된 `birdType`을 `users.bird_type`에 저장한다.
 3. 제출 상태를 `COMPLETED`로 변경한다.
 4. `users.onboardingCompleted=true`로 변경한다.
- 동점인 경우 다음을 처리한다.
 1. 답변과 성향별 점수를 저장한다.
 2. 제출 상태를 `PENDING_TIE_BREAKER`로 저장한다.
 3. 동점 성향 목록을 제출 레코드에 저장한다.
 4. `users.bird_type`을 배정하지 않는다.
 5. `users.onboardingCompleted=false`를 유지한다.
 6. `202 Accepted`와 `submissionId`, `tiedTraits`를 반환한다.
- Tie-breaker 요청에서는 사용자가 선택한 성향이 해당 제출의 `tiedTraits`에 포함되는지 검증한다.
- Tie-breaker 완료 시 파트너 새 배정과 `onboardingCompleted=true`변경을 하나의 트랜잭션으로 처리한다.
- 한 번 완료된 제출은 다시 선택하거나 재응시할 수 없다.
- 새 이미지 파일은 백엔드에서 관리하지 않고 프론트 앱 내부 에셋을 사용한다.

Step 1 입력 / Request DTO

- `testVersion: String`
- `answers: List<PersonalityAnswerRequest>`
 - `questionId: Long`
 - `optionId: Long`

Step 1 출력 / Response DTO

- **단독 1등:** `200 OK`
 - `submissionId: Long`
 - `resultStatus: COMPLETED`
 - `onboardingCompleted: true`
 - `birdType: BirdType`
 - `birdName: String`
 - `trait: String`
 - `description: String`
- **동점:** `202 Accepted`
 - `submissionId: Long`
 - `resultStatus: TIE_BREAKER_REQUIRED`
 - `onboardingCompleted: false`
 - `birdType: null`
 - `tiedTraits: List<PersonalityTrait>`

Step 2 입력 / Request DTO

- DTO명은 `PersonalityTieBreakerRequest` 다.
- `submissionId: Long`
- `selectedTrait: GOURMET | REST | PHOTO | ACTIVITY | CULTURE`

Step 2 출력 / Response DTO

- `200 OK`
- `submissionId: Long`
- `resultStatus: COMPLETED`
- `onboardingCompleted: true`
- `birdType: BirdType`
- `birdName: String`
- `trait: String`
- `description: String`

예외사항 및 검증 로직

- 답변 수가 8개와 다르면 `400 INCOMPLETE_PERSONALITY_TEST` 를 반환한다.
- 동일 질문에 중복 답변하면 `400 DUPLICATED_PERSONALITY_ANSWER` 를 반환한다.
- 유효하지 않은 질문·선택지는 `400 INVALID_PERSONALITY_OPTION` 이다.
- 동점이 아닌 제출에 Tie-breaker를 요청하면 `409 TIE_BREAKER_NOT_REQUIRED` 다.
- `selectedTrait` 이 저장된 동점 후보 목록에 없으면 `400 INVALID_TIE_BREAKER_SELECTION` 이다.
- 이미 Tie-breaker가 완료되었거나 테스트를 완료한 사용자의 재제출은 `409 PERSONALITY_TEST_ALREADY_COMPLETED` 다.
- 다른 사용자의 `submissionId` 에 접근하면 `403 PERSONALITY_SUBMISSION_ACCESS_DENIED` 다.

연동 API 엔드포인트

- Step 1: `POST /api/onboarding/personality-test/submissions`
- Step 2: `POST /api/onboarding/personality-test/submissions/tie-breaker`

추가 확정이 필요한 데이터

- `[추가 확정 필요] GOURMET, REST, PHOTO, ACTIVITY, CULTURE` 각각을 오목눈이·물총새·호반새·딱새·동박새 중 어떤 `birdType` 과 연결할지에 대한 1:1 매핑표는 제공된 결정사항에 포함되어 있지 않다.

3.4 파트너 새

3.4.1 기능명: 파트너 새 조회

기능 설명

- 로그인한 사용자의 파트너 새 종류, 이름과 성향 설명을 조회한다.
- 온보딩 전에도 `204 No Content` 를 사용하지 않고 `200 OK` 와 `birdType=null` 응답을 반환한다.

세부 로직 및 상태 변화

- JWT 사용자 ID로 `users.bird_type` 을 조회한다.
- 프론트엔드는 `birdType` 에 맞는 앱 내부 이미지 에셋을 표시한다.
- 레벨, 경험치와 성장 단계는 관리하지 않는다.
- 온보딩이 완료되지 않았거나 파트너 새가 아직 배정되지 않았으면 정상 응답으로 모든 파트너 새 상세 필드를 `null` 로 반환한다.

출력 / Response DTO

- `onboardingCompleted: Boolean`
- `birdType: BirdType?`
- `birdName: String?`
- `trait: String?`
- `description: String?`

온보딩 전 응답 예시는 다음과 같다.

```
{
  "onboardingCompleted": false,
  "birdType": null,
  "birdName": null,
  "trait": null,
  "description": null
}
```

예외사항 및 검증 로직

- 온보딩 전 또는 Tie-breaker 대기 상태는 오류가 아니며 `200 OK` 를 반환한다.

- 사용자가 존재하지 않거나 삭제된 경우 `404 USER_NOT_FOUND` 를 반환한다.

연동 API 엔드포인트

- `GET /api/users/me/partner-bird`

3.4.2 기능명: 캐릭터 경험치 및 성장 처리

- 캐릭터 경험치, 레벨업, 성장 단계와 외형 진화는 MVP에서 완전히 제외한다.
- 별도의 경험치 적립 API, 내부 이벤트 핸들러와 성장 관련 테이블을 구현하지 않는다.

3.5 이미지 업로드

3.5.1 기능명: 이미지 업로드 URL 발급

기능 설명

- 일정 장소와 여행 게시글 이미지를 백엔드를 거치지 않고 S3로 직접 업로드하기 위한 Presigned URL을 발급한다.
- 프로필 이미지는 업로드 대상에 포함하지 않는다.
- 사용자가 선택할 수 있는 원본 파일 형식은 JPG, PNG, WEBP이며 원본 선택 파일은 장당 최대 5MB다.
- 원본 파일 자체는 서버 또는 S3에 업로드하지 않고, React Native 앱이 최적화한 최종 WebP 파일만 S3에 업로드한다.

세부 로직 및 상태 변화

- 프론트엔드는 업로드 전에 다음 순서로 이미지를 처리한다.
 1. JPG, PNG, WEBP 여부와 원본 5MB 이하 여부를 확인한다.
 2. GPS 좌표를 포함한 EXIF 메타데이터를 제거한다.
 3. 가로 크기를 최대 1080px로 축소하고 원본 비율을 유지한다.
 4. WebP로 변환하고 최종 파일을 장당 500KB 이하로 압축한다.
 5. 최적화가 완료된 WebP 파일의 메타정보로 Presigned URL을 요청한다.
- 백엔드가 발급하는 Presigned URL은 `image/webp`, 최대 500KB, 가로 1080px 이하의 최종 파일 업로드에만 사용한다.

- S3에는 최적화된 WebP 한 개만 저장하며 JPG·PNG 원본과 별도 파생 이미지는 저장하지 않는다.
- 장소별 이미지는 최대 3장, 게시글 전체 이미지는 최대 10장이다.
- 발급 시 파일을 `PENDING` 으로 저장하고 Presigned URL을 반환한다.
- S3 업로드 완료 확인 후 서버가 실제 객체의 MIME 타입, 파일 시그니처, 용량과 이미지 가로 크기를 검증한다.
- 검증에 성공하면 `UPLOADED` 로 변경한다.
- 일정 장소 또는 게시글에 연결되면 `LINKED` 로 변경한다.
- 24시간 동안 연결되지 않은 `PENDING` , `UPLOADED` 파일은 배치로 삭제한다.
- MVP에서는 서버 측 이미지 리사이징, 원본 보관과 별도 파생본 생성 파이프라인을 구현하지 않는다.
- CDN을 사용하더라도 이미 생성된 WebP 파일의 전송 캐시 역할만 담당하며 이미지 변환 규칙은 프론트엔드가 수행한다.

입력 / Request DTO

- `fileName: String`
- `contentType: image/webp`
- `sizeBytes: Long` — 최대 500KB
- `width: Int` — 최대 1080px
- `height: Int`
- `purpose: POST | TRIP_PLACE`

출력 / Response DTO

- `fileId: Long`
- `uploadUrl: String`
- `objectKey: String`
- `expiresAt: LocalDateTime`

예외사항 및 검증 로직

- 최종 업로드 파일이 WebP가 아니면 `400 UNSUPPORTED_FILE_TYPE` 이다.

- 최종 업로드 파일이 500KB를 초과하면 `413 FILE_TOO_LARGE` 를 반환한다.
- 가로 크기가 1080px를 초과하면 `422 IMAGE_DIMENSION_LIMIT_EXCEEDED` 를 반환한다.
- S3 객체의 실제 정보가 발급 요청 메타정보와 다르면 `409 FILE_METADATA_MISMATCH` 를 반환하고 해당 객체를 삭제한다.
- 목적별 첨부 개수 초과는 `422 IMAGE_LIMIT_EXCEEDED` 다.
- 완료 API는 멍등하게 처리한다.

연동 API 엔드포인트

- `POST /api/files/presigned-uploads`
- `POST /api/files/{fileId}/complete`

3.6 장소

3.6.1 기능명: 장소 검색

기능 설명

- 로그인 사용자는 장소명 또는 지역명으로 국내 장소를 검색할 수 있다.
- 장소 검색은 홈 통합 검색, 일정 장소 추가, 일정 위시리스트와 저장 장소 기능에서 공통으로 사용한다.
- 외부 장소 검색 데이터는 네이버 API를 사용한다.
- 네이버 카테고리 원본값은 그대로 보존하고 여행 서비스용 대분류로 정규화한다.
- 국내 장소만 검색 대상으로 사용한다.
- MVP에서는 앱 전용 커스텀 대분류를 추가하지 않고 아래 고정 `PlaceCategory` 만 사용한다.
 - `FOOD`
 - `CAFE`
 - `ATTRACTION`
 - `CULTURE_ART`
 - `ACTIVITY`
 - `NATURE`

- SHOPPING
- ACCOMMODATION
- OTHER

세부 로직 및 상태 변화

- JWT 인증을 필수로 검증한다.
- 검색어를 검증한 후 네이버 장소 검색 API를 호출한다.
- 네이버 응답을 `PlaceSummaryResponse` 로 정규화한다.
 - 네이버 장소 식별값 → `externalPlaceId`
 - `title` → `name`
 - `category` → `externalCategory`
 - `roadAddress` 가 있으면 우선 사용하고 없으면 `address` → `address`
 - `mapx` , `mapy` → `longitude` , `latitude`
- 장소명의 HTML 태그를 제거한다.
- 원본 카테고리 전체 문자열은 `externalCategory` 에 보존한다.
- 여행 서비스용 대분류는 `category: PlaceCategory` 로 별도 반환한다.
- 여행과 관련성이 낮은 병원, 학원, 행정기관, 일반 사무시설 등의 장소는 검색 결과에서 제외한다.
- 여행 관련 장소이지만 고정 대분류에 명확하게 매핑되지 않으면 `OTHER` 로 반환한다.
- MVP에서는 `PHOTO_SPOT` , `NIGHT_VIEW` , `BEACH` , `MARKET` 등의 새 대분류를 추가하지 않는다. 이러한 특성은 필요 시 추후 태그 체계로 확장한다.
- 네이버 원본 카테고리 문자열의 각 depth를 정규화한 뒤 특정 키워드가 포함되어 있는지 `Contains` 방식으로 검사하여 고정 대분류에 매핑한다.
- 매핑 조건은 애플리케이션의 분산된 `if/else` 코드에 하드코딩하지 않고 `place_category_mapping_rules` 테이블에서 우선순위, 포함 키워드, 대상 카테고리 및 제외 여부를 관리한다.
- 한 장소가 여러 규칙에 일치할 수 있으므로 `priority ASC` 기준으로 가장 먼저 일치한 규칙 하나를 적용한다.
- MVP 카테고리 키워드 규칙은 다음과 같다.
 1. 카페 , 다방 , 베이커리 포함 → `CAFE`

2. `음식점` 포함이며 위 CAFE 규칙에 해당하지 않음 → `FOOD`
 3. `숙박`, `펜션`, `호텔`, `모텔` 포함 → `ACCOMMODATION`
 4. `문화`, `예술`, `전시`, `박물관` 포함 → `CULTURE_ART`
 5. `스포츠`, `레저`, `액티비티` 포함 → `ACTIVITY`
 6. `쇼핑`, `마트`, `시장` 포함 → `SHOPPING`
 7. `자연`, `산`, `바다`, `계곡` 포함 → `NATURE`
 8. `관광`, `명소`, `공원` 포함 → `ATTRACTION`
- 여행 관련 장소지만 위 규칙에 매핑되지 않으면 `OTHER` 로 반환한다.
 - 병원, 행정기관, 학원, 일반 사무시설 등 여행과 직접 관련 없는 키워드가 포함된 장소는 `OTHER` 로 노출하지 않고 검색 결과에서 제외한다.
 - 내부 DB에 이미 존재하는 장소면 `placeId` 를 반환하고 사용자의 저장 여부를 `saved` 로 반환한다.
 - 검색만으로 장소를 DB에 저장하지 않는다.
 - 저장, 일정 추가 또는 위시리스트 추가 시점에 내부 `places` 를 생성한다.
 - `provider + externalPlaceId` 에 유니크 제약을 적용한다.
 - 네이버 무료 일일 호출 한도를 초과하면 유료 과금으로 전환하지 않고 당일 검색을 차단한다.

입력 / Request DTO

- `query: String`
- `latitude: BigDecimal?`
- `longitude: BigDecimal?`
- `category: PlaceCategory?`
- `start: Int = 1`
- `display: Int = 15`

출력 / Response DTO

- `items`
 - `placeId: Long?`

- `externalPlaceId: String`
- `name: String`
- `externalCategory: String`
- `category: PlaceCategory`
- `address: String`
- `latitude: BigDecimal`
- `longitude: BigDecimal`
- `saved: Boolean`
- `total: Int`
- `isEnd: Boolean`

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED` 다.
- 검색어가 비어 있으면 `400 SEARCH_QUERY_REQUIRED` 다.
- 검색어가 2자 미만이면 `400 SEARCH_QUERY_TOO_SHORT` 다.
- 고정 대분류에 없는 값은 `400 INVALID_PLACE_CATEGORY` 다.
- 잘못된 좌표는 `400 INVALID_COORDINATES` 다.
- 네이버 API 장애는 `503 PLACE_SEARCH_UNAVAILABLE` 이다.
- 일일 무료 호출 한도 초과는 `503 PLACE_SEARCH_DAILY_LIMIT_EXCEEDED` 이며 사용자 메시지는 `금일 장소 검색 한도가 초과되었습니다.` 로 반환한다.

연동 API 엔드포인트

- `GET /api/places/search`

3.6.2 기능명: 장소 상세 조회

기능 설명

- 로그인 사용자는 장소명, 주소, 카테고리과 좌표를 조회한다.
- 사용자의 장소 저장 여부를 함께 반환한다.

- 내부에 존재하지만 폐업 또는 비활성으로 확인된 장소도 삭제된 장소처럼 숨기지 않고 `200 OK + status=CLOSED` 로 반환한다.

세부 로직 및 상태 변화

- JWT 인증을 필수로 검증한다.
- 내부 DB 장소정보를 우선 조회한다.
- 필요할 때 네이버 API의 최신 정보로 보완한다.
- `externalCategory` 와 정규화된 `category` 를 함께 반환한다.
- 저장 관계를 조회하여 `saved` 를 반환한다.
- 외부 정보 또는 운영 데이터에서 폐업이 확인되면 `places.status=CLOSED` 로 갱신한다.
- `CLOSED` 장소는 기존 장소명, 주소, 좌표와 저장 여부를 반환하되 응답 상태 필드로 폐업 사실을 명확히 전달한다.

출력 / Response DTO

- `placeId: Long`
- `status: ACTIVE | CLOSED`
- `name: String`
- `externalCategory: String`
- `category: PlaceCategory`
- `address: String`
- `regionCode: String?`
- `latitude: BigDecimal`
- `longitude: BigDecimal`
- `imageUrl: String?`
- `saved: Boolean`

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED` 다.
- 내부 DB에 존재하지 않는 장소는 `404 PLACE_NOT_FOUND` 다.

- 폐업·비활성 장소는 오류로 처리하지 않고 `200 OK` 와 `status=CLOSED` 를 반환한다.

연동 API 엔드포인트

- `GET /api/places/{placeId}`

3.6.3 기능명: 장소 저장·취소·목록·메모 관리

기능 설명

- 사용자는 장소를 저장하거나 저장 취소할 수 있다.
- 저장 장소는 폴더·지역 분류 없이 최신 저장순 일렬 목록으로 제공한다.
- 저장 시점에는 메모를 입력받지 않고 저장 이후 별도로 작성·수정한다.

세부 로직 및 상태 변화

- 외부 검색 결과가 아직 내부 DB에 없으면 저장 시점에 `places` 를 생성한다.
- `(userId, placeId)` 저장 관계 중복을 방지한다.
- 저장·취소 요청은 멍등하게 처리한다.
- 목록은 저장일 내림차순과 커서 페이지네이션을 사용한다.
- 저장 장소 메모는 사용자 본인만 조회·수정할 수 있다.
- 메모는 공백을 포함하여 최대 100자다.
- 저장 관계를 취소하면 해당 개인 메모도 함께 삭제한다.

입력 / Request DTO

- 저장·취소: Path `placeId: Long`
- 목록: `cursor: Long?` , `size: Int = 20`
- 메모 수정: `memo: String?`

출력 / Response DTO

- 저장·취소 성공: `204 No Content`
- 목록

- `placeId`, `name`, `category`, `thumbnailUrl`, `memo`, `savedAt`
- `nextCursor`
- 메모 수정 성공: `placeId`, `memo`, `updatedAt`

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED` 다.
- 존재하지 않는 장소는 `404 PLACE_NOT_FOUND` 다.
- 메모가 100자를 초과하면 `400 SAVED_PLACE_MEMO_TOO_LONG` 이다.

연동 API 엔드포인트

- `PUT /api/users/me/saved-places/{placeId}`
- `DELETE /api/users/me/saved-places/{placeId}`
- `GET /api/users/me/saved-places`
- `PATCH /api/users/me/saved-places/{placeId}/memo`

3.7 여행 일정

3.7.1 기능명: 직접 일정 생성

기능 설명

- 사용자는 단일 지역, 날짜, 단일 동행 유형, 최대 3개 테마와 일정 선호도를 선택한다.
- 제목은 시스템이 자동 생성한다.
- 직접 일정은 Day별 장소가 없는 빈 일정으로 생성된다.
- 여행 기간은 최대 7박 8일이다.
- 완료된 여행도 수정할 수 있다.
- 다만 발행된 Post가 연결된 Trip은 Post가 삭제될 때까지 경로 구조 수정이 잠기며, 경로와 무관한 정보만 수정할 수 있다.

세부 로직 및 상태 변화

- Trip과 여행 날짜 수만큼의 TripDay를 하나의 트랜잭션으로 생성한다.

- `sourceType=MANUAL` 로 저장한다.
- 공개 범위 기본값은 `PRIVATE` 다.
- 일정 진행 상태는 날짜를 기준으로 계산한다.
 - `UPCOMING` : 현재 날짜가 `startDate` 보다 이전
 - `IN_PROGRESS` : `startDate ≤ 현재 날짜 ≤ endDate`
 - `COMPLETED` : 현재 날짜가 `endDate` 보다 이후
- 취소 여부는 별도 상태값을 저장하지 않고 `cancelledAt: LocalDateTime?` 으로 관리한다.
- `cancelledAt` 이 있으면 날짜 판정 결과보다 우선하여 유효 상태를 `CANCELLED` 로 반환한다.
- `companionType` 은 단일 Enum으로 저장한다.
- `themes` 는 최소 1개, 최대 3개다.
- 각 Day의 `dayNumber` 는 1부터 순차 부여한다.
- 신규 Trip에는 발행 Post가 없으므로 `routeEditable=true` 다.

입력 / Request DTO

- `regionCode: String`
- `startDate: LocalDate`
- `endDate: LocalDate`
- `companionType: CompanionType`
- `themes: Set<TravelTheme>`
- `pace: DENSE | RELAXED`

출력 / Response DTO

- `tripId` , `title` , `sourceType`
- `status: UPCOMING | IN_PROGRESS | COMPLETED | CANCELLED`
- `cancelledAt: LocalDateTime?`
- `visibility: PRIVATE`
- `region` , `startDate` , `endDate`

- `companionType` , `themes` , `pace` , `hashtags` , `days`
- `routeEditable: true`
- `routeLockedReason: null`

예외사항 및 검증 로직

- 날짜 누락은 `400 TRIP_DATE_REQUIRED` 다.
- 종료일이 시작일보다 빠르면 `400 INVALID_TRIP_PERIOD` 다.
- 8일 초과는 `400 TRIP_PERIOD_TOO_LONG` 이다.
- 지역 누락·복수 지역은 `400 REGION_REQUIRED` , `400 MULTIPLE_REGIONS_NOT_ALLOWED` 다.
- 동행 유형 오류는 `400 INVALID_COMPANION_TYPE` 이다.
- 테마 누락·3개 초과는 `400 THEME_REQUIRED` , `400 TOO_MANY_THEMES` 다.
- 일정 선호도 오류는 `400 INVALID_TRIP_PACE` 다.

연동 API 엔드포인트

- `POST /api/trips`

3.7.2 기능명: 일정 상세 조회

기능 설명

- 일정 기본 정보와 Day별 장소 목록을 조회한다.
- 지도에는 전체 또는 선택 Day의 번호 핀과 단순 연결선을 표시한다.
- 공개 범위는 `PUBLIC` , `MEMO_PRIVATE` , `PRIVATE` 다.
- 소유자는 모든 내용을 조회할 수 있다.
- 비로그인 사용자도 `PUBLIC` , `MEMO_PRIVATE` 일정을 조회할 수 있다.
- 미확정 AI 결과는 `previewId` 기반 미리보기 API에서 조회한다.
- 일정 소유자에게 현재 Trip의 경로 수정 가능 여부와 잠금 원인이 함께 제공된다.

세부 로직 및 상태 변화

- `PUBLIC` 은 일정, 장소, 메모와 사진을 모두 반환한다.

- `MEMO_PRIVATE` 은 일정, 장소와 사진을 반환하고 장소 메모만 숨긴다.
- 비소유자에게 `memo=null` , `memoMasked=true` 를 반환한다.
- `PRIVATE` 은 소유자만 조회할 수 있으며 비소유자에게 내용을 반환하지 않는다.
- 일정 상태는 조회 시 날짜 기준으로 계산한다.
- 다만 `cancelledAt` 이 존재하면 날짜와 무관하게 `status=CANCELLED` 로 반환한다.
- 취소된 Trip은 읽기 전용이므로 소유자 응답에도 `routeEditable=false` , `contentEditable=false` , `routeLockedReason=TRIP_CANCELLED` 를 반환한다.
- 공개 범위 기본값은 `PRIVATE` 다.
- AI 작업 상태는 Trip 상태와 구분한다.
- 이동 경로는 장소 좌표와 순서를 이용한 단순 직선·점선 연결로 표시한다.
- `publishedAt IS NOT NULL` 이고 `deletedAt IS NULL` 인 Post가 Trip에 연결되어 있으면 경로 잠금 상태로 판단한다.
- 발행 Post의 공개 범위가 `PUBLIC` , `MEMO_PRIVATE` , `PRIVATE` 중 무엇이든 잠금을 유지하며, 운영자가 `BLOCKED` 로 변경해도 Post가 삭제되지 않은 동안 잠금을 유지한다.
- DRAFT Post만 존재하면 경로 잠금을 적용하지 않는다.

출력 / Response DTO

- `tripId` , `owner` , `sourceType` , `status` , `visibility`
- `cancelledAt: LocalDateTime?`
- `title` , `summary` , `region` , `startDate` , `endDate`
- `pace` , `companionType` , `themes` , `hashtags`
- `routeEditable: Boolean`
- `routeLockedReason: PUBLISHED_POST_EXISTS | TRIP_CANCELLED | null`
- `publishedPostId: Long?`
- `contentEditable: Boolean`
- `days`
 - `tripPlaceId` , `placeId` , `order`
 - `plannedTime` , `durationMinutes`
 - `memo` , `memoMasked` , `images` , `reason` , `coordinates`

- 비소유자 응답에서는 편집 기능이 없으므로 `routeEditable=false`, `contentEditable=false` 를 반환하고 잠금 원인과 연결 Post ID는 노출하지 않는다.

예외사항 및 검증 로직

- 존재하지 않는 일정은 `404 TRIP_NOT_FOUND` 다.
- `PRIVATE` 일정의 비소유자·비로그인 접근은 `404 TRIP_NOT_FOUND` 다.
- `MEMO_PRIVATE` 비소유자 응답에서는 메모를 반환하지 않는다.

연동 API 엔드포인트

- `GET /api/trips/{tripId}`

3.7.3 기능명: 일정 기본 정보 수정

기능 설명

- 일정 소유자는 예정·진행·완료 여행을 수정할 수 있다.
- 발행된 Post가 연결되지 않은 Trip은 경로 정보와 비경로 정보를 모두 수정할 수 있다.
- 발행된 Post가 연결된 Trip은 제목, 설명, 동행 유형, 테마, 일정 밀도와 Trip 공개 범위만 수정할 수 있고 지역·기간·Day 구조는 수정할 수 없다.

세부 로직 및 상태 변화

- 경로 수정 필드는 `regionCode`, `startDate`, `endDate` 다.
- 비경로 수정 필드는 `title`, `summary`, `companionType`, `themes`, `pace`, `visibility` 다.
- `companionType` 은 생성·조회·수정에서 단일값으로 통일한다.
- 경로 변경 요청 전에 `publishedAt IS NOT NULL AND deletedAt IS NULL` 인 연결 Post 존재 여부를 Service 계층에서 검증한다.
- 경로가 잠긴 상태에서 경로 수정 필드가 하나라도 전달되면 전체 요청을 거부하고 비경로 필드도 함께 변경하지 않는다.
- 경로가 잠기지 않은 경우 날짜 증가 시 Day를 추가한다.
- 날짜 축소로 제거될 Day에 장소가 있으면 사용자 확인 없이 삭제하지 않는다.
- 확인되지 않은 요청에는 `409 TRIP_DAY_REMOVAL_CONFIRMATION_REQUIRED` 와 제거 예정 Day·장소 요약을 반환하고, 사용자가 `confirmDayRemoval=true` 로 재요청하면 삭제한다.

- 제거되는 Day의 장소 메모와 사진도 함께 삭제한다.
- `@Version` 낙관적 락으로 동시 수정 충돌을 방지한다.
- 수정 후 진행 상태는 변경된 날짜를 기준으로 다시 계산한다.

입력 / Request DTO

- `title: String?`
- `summary: String?`
- `regionCode: String?`
- `startDate: LocalDate?`
- `endDate: LocalDate?`
- `companionType: CompanionType?`
- `themes: Set<TravelTheme>?`
- `pace: DENSE | RELAXED?`
- `visibility: PUBLIC | MEMO_PRIVATE | PRIVATE?`
- `confirmDayRemoval: Boolean = false`
- `version: Long`

예외사항 및 검증 로직

- 비소유자는 `403 TRIP_ACCESS_DENIED` 다.
- 버전 충돌은 `409 TRIP_MODIFICATION_CONFLICT` 다.
- 잠긴 Trip의 지역·기간·Day 구조 변경은 `409 TRIP_ROUTE_LOCKED_BY_PUBLISHED_POST` 다.
- 완료된 일정이라는 이유만으로 수정 요청을 거부하지 않는다.
- `cancelledAt` 이 존재하는 Trip은 제목, 날짜, 장소, 경로, 메모, 사진, 공개 범위 등 모든 수정 요청을 거부하고 읽기 전용으로 유지한다.
- 취소된 Trip 수정 요청은 `409 TRIP_CANCELLED_READ_ONLY` 를 반환한다.

연동 API 엔드포인트

- `PATCH /api/trips/{tripId}`

3.7.4 기능명: Day별 장소 추가·삭제

기능 설명

- 일정 소유자는 특정 Day에 장소를 추가하거나 삭제한다.
- 발행된 Post가 연결된 Trip에서는 장소 추가·삭제를 허용하지 않는다.

세부 로직 및 상태 변화

- 일정 소유권과 Day 소속을 검증한다.
- `cancelledAt` 이 존재하면 장소 추가·삭제 요청을 `409 TRIP_CANCELLED_READ_ONLY` 로 차단한다.
- 장소 추가·삭제 전에 Trip의 경로 잠금 여부를 검증한다.
- 같은 장소는 동일 Day에 중복 추가할 수 없다.
- 같은 장소를 다른 Day에는 추가할 수 있다.
- Day별 최대 장소 수는 15개다.
- 장소 추가 시 해당 장소가 Trip 위시리스트에 있으면 위시리스트에서 자동 제거한다.
- 따라서 같은 Trip에서 동일 장소는 위시리스트와 Day에 동시에 존재할 수 없다.
- 장소 삭제 시 연결된 메모, 사진 관계와 실제 이미지 파일을 삭제한다.
- 삭제 후 Day 내 순서를 연속되게 재정렬한다.

입력 / Request DTO

- `placeId: Long`
- `order: Int?`
- `plannedTime: LocalTime?`
- `durationMinutes: Int?`

예외사항 및 검증 로직

- 존재하지 않는 Day는 `404 TRIP_DAY_NOT_FOUND` 다.
- 존재하지 않는 장소는 `404 PLACE_NOT_FOUND` 다.
- 동일 Day 중복은 `409 PLACE_ALREADY_ADDED` 다.

- 15개 초과는 `422 TRIP_DAY_PLACE_LIMIT_EXCEEDED` 다.
- 경로 잠금 상태에서는 `409 TRIP_ROUTE_LOCKED_BY_PUBLISHED_POST` 를 반환한다.
- 취소된 Trip은 `409 TRIP_CANCELLED_READ_ONLY` 다.

연동 API 엔드포인트

- `POST /api/trips/{tripId}/days/{dayNumber}/places`
- `DELETE /api/trips/{tripId}/days/{dayNumber}/places/{tripPlaceId}`

3.7.5 기능명: 장소 순서 변경

기능 설명

- Day 안에서 장소 순서를 드래그 방식으로 변경한다.
- 발행된 Post가 연결된 Trip에서는 장소 순서를 변경할 수 없다.

세부 로직 및 상태 변화

- 클라이언트는 Day의 전체 `tripPlaceId` 를 원하는 순서대로 전송한다.
- `cancelledAt` 이 존재하면 순서 변경 요청을 `409 TRIP_CANCELLED_READ_ONLY` 로 차단한다.
- 순서 변경 전에 Trip 경로 잠금 여부를 검증한다.
- 누락, 중복과 다른 Day 소속 ID를 검증한다.
- 하나의 트랜잭션에서 순서를 변경한다.

입력 / Request DTO

- `orderedTripPlaceIds: List<Long>`

예외사항 및 검증 로직

- 누락·중복은 `400 INVALID_PLACE_ORDER_REQUEST` 다.
- 다른 Day 장소는 `400 TRIP_PLACE_DAY_MISMATCH` 다.
- 경로 잠금 상태에서는 `409 TRIP_ROUTE_LOCKED_BY_PUBLISHED_POST` 를 반환한다.
- 취소된 Trip은 `409 TRIP_CANCELLED_READ_ONLY` 다.

연동 API 엔드포인트

- `PUT /api/trips/{tripId}/days/{dayNumber}/place-orders`

3.7.6 기능명: 장소별 메모·사진·일정 부가정보 수정

기능 설명

- 일정 소유자는 각 장소의 한줄 메모, 사진, 추천 방문시간과 체류시간을 추가·수정할 수 있다.
- 이 기능은 장소 ID, Day와 방문 순서를 바꾸지 않으므로 Trip 경로가 잠겨 있어도 사용할 수 있다.

세부 로직 및 상태 변화

- `cancelledAt` 이 존재하면 메모·사진·추천시간·체류시간 수정 요청을 `409 TRIP_CANCELLED_READ_ONLY` 로 차단한다.
- 메모는 공백 포함 최대 100자다.
- 장소별 사진은 최대 3장이다.
- 파일 소유권, 업로드 완료 상태와 목적을 검증한다.
- 기존 목록에서 제거된 이미지 파일은 즉시 삭제한다.
- `plannedTime` 과 `durationMinutes` 는 장소 구성이나 방문 순서를 변경하지 않는 부가정보로 처리한다.
- 요청에서 `placeId` , `dayNumber` , `order` 는 변경할 수 없다.

입력 / Request DTO

- `memo: String?`
- `imageFileIds: List<Long>?`
- `plannedTime: LocalTime?`
- `durationMinutes: Int?`

예외사항 및 검증 로직

- 비소유자는 `403 TRIP_ACCESS_DENIED` 다.

- 메모 초과는 `400 TRIP_PLACE_MEMO_TOO_LONG` 이다.
- 사진 3장 초과는 `422 TRIP_PLACE_IMAGE_LIMIT_EXCEEDED` 다.
- 파일 소유권 오류는 `403 FILE_ACCESS_DENIED` 다.
- 체류시간이 0 이하이면 `400 INVALID_TRIP_PLACE_DURATION` 이다.
- 취소된 Trip은 `409 TRIP_CANCELLED_READ_ONLY` 다.

연동 API 엔드포인트

- `PATCH /api/trips/{tripId}/places/{tripPlaceId}/content`

3.7.7 기능명: 일정 위시리스트

기능 설명

- Trip에 종속된 위시리스트 장소를 관리한다.
- 위시리스트는 AI 경로 배치에 사용할 수 있다.
- 발행된 Post가 연결된 Trip에서도 위시리스트 자체의 추가·삭제·조회는 가능하지만, 위시리스트 장소를 Day에 배정하거나 AI 결과를 Trip에 적용하는 경로 변경은 허용하지 않는다.

세부 로직 및 상태 변화

- `(tripId, placeId)` 중복을 방지한다.
- 일정 소유자만 수정한다.
- `cancelledAt` 이 존재하면 위시리스트 조회만 허용하고 추가·삭제·Day 배정 요청은 `409 TRIP_CANCELLED_READ_ONLY` 로 차단한다.
- 경로가 잠기지 않은 Trip에서 Day에 장소를 배정하면 동일 장소를 위시리스트에서 자동 제거한다.
- 위시리스트와 Day에 동일 장소가 동시에 존재할 수 없다.

입력 / Request DTO

- `placeId: Long`

예외사항 및 검증 로직

- 비소유자는 `403 TRIP_ACCESS_DENIED` 다.
- 중복 추가는 멍등 처리한다.
- 이미 Day에 배정된 장소를 위시리스트에 추가하면 `409 PLACE_ALREADY_ASSIGNED_TO_DAY` 를 반환한다.
- 취소된 Trip의 위시리스트 변경은 `409 TRIP_CANCELLED_READ_ONLY` 다.

연동 API 엔드포인트

- `POST /api/trips/{tripId}/wishlist-places`
- `GET /api/trips/{tripId}/wishlist-places`
- `DELETE /api/trips/{tripId}/wishlist-places/{placeId}`

3.7.8 기능명: 내 여행 목록 및 캘린더

기능 설명

- 내 여행을 예정·진행·완료로 구분하여 조회한다.
- 확정 저장된 Trip만 목록과 캘린더에 표시한다.
- `cancelledAt IS NOT NULL` 인 여행은 상태가 `CANCELLED` 이며 내 여행 목록과 캘린더에서 표시하지 않는다.

세부 로직 및 상태 변화

- 상태 판정은 `Asia/Seoul` 현재 날짜와 여행 날짜를 기준으로 한다.
 - `UPCOMING` : 오늘 < 시작일
 - `IN_PROGRESS` : 시작일 ≤ 오늘 ≤ 종료일
 - `COMPLETED` : 종료일 < 오늘
- 종료일이 지나면 별도 버튼 없이 자동으로 완료로 판정한다.
- `CANCELLED` 일정은 목록과 캘린더에서 제외한다.
- `MANUAL` , `AI` Trip을 함께 조회한다.
- 미확정 preview와 AI job은 포함하지 않는다.
- 커서 페이지네이션을 적용한다.

입력 / Request DTO

- 목록: `category: UPCOMING | IN_PROGRESS | COMPLETED`, `cursor`, `size`
- 캘린더: `from`, `to`

예외사항 및 검증 로직

- 잘못된 기간은 `400 INVALID_DATE_RANGE` 다.
- AI `jobId`, `previewId` 는 내 여행 목록 식별자로 사용할 수 없다.

연동 API 엔드포인트

- `GET /api/users/me/trips`
- `GET /api/users/me/travel-calendar`

3.7.9 기능명: 여행 취소

기능 설명

- 일정의 진행 상태는 날짜로 계산하되, 사용자가 여행을 취소하면 `cancelledAt` 에 취소 일시를 기록한다.
- `cancelledAt` 이 존재하는 Trip은 여행 날짜와 관계없이 항상 `CANCELLED` 로 판정한다.
- 취소는 상태를 되돌리는 임시 기능이 아니라 해당 Trip을 영구적인 읽기 전용 상태로 전환하는 기능이다.

세부 로직 및 상태 변화

- JWT 사용자 ID와 Trip 소유자를 검증한다.
- 취소 요청 시 Trip 행을 잠그고 현재 `cancelledAt` 과 연결 Post를 확인한다.
- `publishedAt IS NOT NULL AND deletedAt IS NULL` 인 Post가 연결되어 있으면 Trip을 취소할 수 없다.
- 발행 Post가 연결된 경우 사용자가 먼저 해당 Post를 완전히 삭제해야 하며, `PRIVATE` 전환이나 운영 `BLOCKED` 처리는 취소 허용 조건이 아니다.
- 발행 Post가 없으면 `cancelledAt=현재시각` 을 기록한다.

- 취소 후 날짜, 지역, Day, 장소, 순서, 위시리스트, 메모, 사진, 공개 범위, 제목과 기타 부가정보를 포함한 모든 수정 요청을 차단한다.
- 취소된 Trip은 내 여행 목록과 캘린더에서 제외하지만 소유자는 직접 상세 조회를 통해 읽기 전용으로 확인할 수 있다.
- 취소 해제 또는 복구 API는 제공하지 않는다.
- 이미 취소된 Trip에 대한 반복 취소 요청은 멍등하게 `204 No Content` 를 반환한다.

입력 / Request DTO

- Path `tripId: Long`
- 별도 Request Body 없음

출력 / Response DTO

- 성공 시 `204 No Content`

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED` 다.
- 존재하지 않는 Trip은 `404 TRIP_NOT_FOUND` 다.
- 비소유자는 `403 TRIP_ACCESS_DENIED` 다.
- 발행된 Post가 연결된 Trip은 `409 TRIP_CANCEL_REQUIRES_POST_DELETION` 을 반환하고 연결된 `postId` 를 오류 데이터에 포함한다.
- 취소된 Trip의 수정 API 요청은 `409 TRIP_CANCELLED_READ_ONLY` 다.

연동 API 엔드포인트

- `[AI 제안·구현 기준] POST /api/trips/{tripId}/cancel`

3.8 여행 기록

3.8.1 기능명: 여행 기록 작성 및 임시 저장

기능 설명

- 사용자는 Trip을 기반으로 여행 게시글을 임시 저장하거나 발행한다.

- Trip과 Post는 별도 객체다.
- 하나의 Trip에는 삭제되지 않은 Post를 하나만 둘 수 있다.
- 본문은 일반 텍스트만 지원한다.
- 제목은 최대 50자, 본문은 최대 2,000자다.
- 해시태그는 게시글당 최대 5개이며 각 태그는 최대 10자다.
- DRAFT Post는 Trip 경로를 잠그지 않으며, Post가 최초 발행되어 `publishedAt` 이 설정되는 순간부터 Trip 경로를 잠근다.

세부 로직 및 상태 변화

- Post는 `tripId` 를 참조한다.
- 임시 저장은 `status=DRAFT` , 발행은 `status=PUBLISHED` 로 저장한다.
- 강제 숨김 시 운영자가 `status=BLOCKED` 로 변경한다.
- `DRAFT` 는 작성자만 조회할 수 있고 커뮤니티·검색·포토맵·통계에서 제외한다.
- 제목, 본문, 이미지, 장소, 해시태그와 공개 범위를 하나의 트랜잭션으로 저장한다.
- 게시글 이미지는 최대 10장이다.
- 제목, 본문과 해시태그 길이·개수 제한은 DRAFT와 발행 요청 모두에 적용한다.
- DB 컬럼은 `posts.title VARCHAR(50)` , `posts.content TEXT` , 해시태그명은 `VARCHAR(10)` 기준으로 설계한다.
- DRAFT에서 최초 발행할 때 Trip 행을 잠그고 `publishedAt` 을 저장하여 Post 발행과 동시 Trip 경로 변경 경쟁 조건을 방지한다.
- 경로 잠금 판정 기준은 연결 Post의 `publishedAt IS NOT NULL AND deletedAt IS NULL` 이다.
- 발행 후 `visibility` 가 `PRIVATE` 로 변경되거나 `status=BLOCKED` 가 되어도 Post가 삭제되지 않은 동안 경로 잠금을 유지한다.
- 삭제된 Post와 같은 Trip으로 새 Post를 작성할 수 있으므로 단순 `UNIQUE(posts.trip_id)` 를 사용하지 않는다.
- [AI 제안·구현 기준] MySQL에서는 `deletedAt IS NULL` 인 Post만 유일하도록 생성 컬럼 `active_trip_id` 와 유니크 인덱스를 사용하거나, Trip 행 잠금과 Service 검증으로 동일 조건을 보장한다.

- Trip 공개 범위는 일정 상세 접근을, Post 공개 범위는 커뮤니티 게시글 접근을 각각 제어하며 자동 동기화하지 않는다.

입력 / Request DTO

- `tripId: Long`
- `title: String`
- `content: String`
- `representativeFileId: Long?`
- `imageFileIds: List<Long>`
- `placeIds: List<Long>`
- `hashtags: List<String>`
- `visibility: PUBLIC | MEMO_PRIVATE | PRIVATE`
- `publish: Boolean`

출력 / Response DTO

- `postId`
- `status: DRAFT | PUBLISHED`
- `visibility`
- `publishedAt`
- `createdAt`
- `tripRouteLocked: Boolean`

예외사항 및 검증 로직

- 제목·본문 누락은 발행 요청에서 `400 INVALID_POST_CONTENT` 다.
- 제목이 50자를 초과하면 `400 POST_TITLE_TOO_LONG` 이다.
- 본문이 2,000자를 초과하면 `400 POST_CONTENT_TOO_LONG` 이다.
- 해시태그가 5개를 초과하면 `400 POST_HASHTAG_LIMIT_EXCEEDED`, 개별 태그가 10자를 초과하면 `400 POST_HASHTAG_TOO_LONG` 이다.
- 동일 Trip에 삭제되지 않은 Post가 있으면 `409 POST_ALREADY_EXISTS_FOR_TRIP` 이다.

- 타인 Trip은 `403 TRIP_ACCESS_DENIED` 다.
- 이미지 10장 초과는 `422 POST_IMAGE_LIMIT_EXCEEDED` 다.
- Post 발행과 Trip 경로 수정이 충돌하면 한 요청만 성공하도록 트랜잭션 락을 적용한다.

연동 API 엔드포인트

- `POST /api/posts`

3.8.2 기능명: 게시글 공개 범위 설정

기능 설명

- `PUBLIC` 은 경로, 장소, 장소 메모, 장소 사진과 본문을 공개한다.
- `MEMO_PRIVATE` 은 게시글 본문과 경로·장소·사진을 공개하고 장소 메모만 숨긴다.
- `PRIVATE` 은 작성자만 조회할 수 있으며 공유 링크로도 접근할 수 없다.
- `MEMO_PRIVATE` 게시글은 커뮤니티 목록에 노출한다.

세부 로직 및 상태 변화

- 공개 범위는 Post의 `visibility` 로 저장한다.
- 비작성자에게 `MEMO_PRIVATE` 장소 메모는 `null`, `memoMasked=true` 로 반환한다.
- `contentMasked` 는 사용하지 않는다.
- 장소 사진은 `MEMO_PRIVATE` 에서도 공개한다.
- `PRIVATE` 게시글은 커뮤니티, 검색, 홈 추천에서 제외한다.

예외사항 및 검증 로직

- `PRIVATE` 비작성자 접근은 `404 POST_NOT_FOUND` 다.
- `BLOCKED` 게시글 접근도 `404 POST_NOT_FOUND` 다.

3.8.3 기능명: 게시글 목록·상세 조회

기능 설명

- 작성자는 자신의 DRAFT·PUBLISHED 게시글 목록을 조회한다.

- 비로그인 사용자는 `PUBLIC`, `MEMO_PRIVATE` 로 발행된 게시물 상세를 조회할 수 있다.

세부 로직 및 상태 변화

- 내 목록에는 공개 범위와 관계없이 삭제되지 않은 자신의 게시글을 포함한다.
- 마이페이지 기록 수에서는 DRAFT를 제외한다.
- 일반 상세는 `status=PUBLISHED` 이며 `visibility IN (PUBLIC, MEMO_PRIVATE)` 인 게시글만 비작성자에게 허용한다.
- `status=BLOCKED` 또는 삭제된 게시글은 일반 사용자에게 반환하지 않는다.
- 상세 조회와 조회수 집계는 분리한다.

출력 / Response DTO

- 목록: `postId`, `tripId`, `title`, `thumbnailUrl`, `region`, `status`, `visibility`, `createdAt`
- 상세: 작성자, 제목, 본문, 이미지, 장소, 경로, 공개 범위와 통계
- 작성자 정보에 `birdType` 을 포함한다.

예외사항 및 검증 로직

- 존재하지 않거나 접근 불가능하면 `404 POST_NOT_FOUND` 다.
- 비로그인 사용자가 DRAFT·PRIVATE·BLOCKED 게시글을 요청하면 `404` 다.

연동 API 엔드포인트

- `GET /api/users/me/posts`
- `GET /api/posts/{postId}`

3.8.4 기능명: 여행 기록 수정·삭제

기능 설명

- 작성자만 게시글을 수정·삭제할 수 있다.
- 완료 여행에 연결된 게시글도 수정할 수 있다.
- 발행 이후에도 제목 오타, 본문, 대표 이미지, 첨부 이미지, 공개 범위와 해시태그는 수정할 수 있다.

- 발행 이후에는 Post의 연결 Trip, 게시 경로 장소 목록과 방문 순서를 변경할 수 없다.

세부 로직 및 상태 변화

- DRAFT Post는 제목, 본문, 이미지, 장소, 해시태그와 공개 범위를 수정할 수 있다.
- PUBLISHED 또는 BLOCKED Post는 제목, 본문, 대표 이미지, 첨부 이미지, 해시태그와 공개 범위만 수정할 수 있다.
- 발행 Post의 `placeIds`, 연결 `tripId`, Day·장소 순서를 변경하는 요청은 거부한다.
- 공개 범위를 `PRIVATE`로 변경해도 원본 Trip 경로 잠금은 해제하지 않는다.
- 운영자가 `BLOCKED`로 변경해도 원본 Trip 경로 잠금은 해제하지 않는다.
- 삭제 시 연결된 S3 이미지 파일과 DB 파일 메타데이터를 즉시 삭제한다.
- 사용자 관점에서는 Post가 완전히 삭제되며 커뮤니티, 검색, 홈 추천과 포토맵에서 즉시 제외된다.
- 포토맵 방문 정보는 해당 게시글 삭제 즉시 재집계된다.
- 타 사용자의 저장 관계는 유지하되 `sourceAvailable=false`로 변경하고 원본 내용을 노출하지 않는다.
- 삭제 완료 시 `deletedAt`을 기록하여 원본 Trip의 경로 잠금을 해제한다.
- 이후 같은 Trip을 수정하여 새 Post를 발행해도 과거 삭제 Post를 참조한 `SavedRoute`는 새 Post에 자동 연결하지 않는다.
- 참조 무결성과 과거 `SavedRoute` 식별을 위해 삭제 Post 행은 tombstone 형태로 남길 수 있으며, 제목·본문·이미지 등 사용자 콘텐츠는 제거한다.

입력 / Request DTO

- `title: String?`
- `content: String?`
- `representativeFileId: Long?`
- `imageFileIds: List<Long>?`
- `placeIds: List<Long>?` — DRAFT에서만 허용
- `hashtags: List<String>?`
- `visibility: PUBLIC | MEMO_PRIVATE | PRIVATE?`
- `version: Long`

예외사항 및 검증 로직

- 비작성자는 `403 POST_ACCESS_DENIED` 다.
- 버전 충돌은 `409 POST_MODIFICATION_CONFLICT` 다.
- 발행 Post의 경로 관련 필드 변경은 `409 POST_ROUTE_LOCKED_AFTER_PUBLISH` 다.
- 수정 요청에도 제목 50자, 본문 2,000자, 해시태그 5개·각 10자 제한을 동일하게 적용한다.
- 이미 삭제된 요청은 멍등 `204` 로 처리한다.

연동 API 엔드포인트

- `PATCH /api/posts/{postId}`
- `DELETE /api/posts/{postId}`

3.9 커뮤니티

3.9.1 기능명: 전체·인기·이웃새 게시물 조회

기능 설명

- `ALL` 은 전체 공개 가능한 게시글을 최신순으로 조회한다.
- `POPULAR` 은 기간별 인기 점수를 기준으로 조회한다.
- `FOLLOWING` 은 로그인 사용자가 팔로우한 사용자의 글을 조회한다.
- 비로그인 사용자는 `ALL` , `POPULAR` 을 조회할 수 있다.
- `MEMO_PRIVATE` 게시글을 목록에 포함한다.

세부 로직 및 상태 변화

- 조회 대상은 `status=PUBLISHED` , `visibility IN (PUBLIC, MEMO_PRIVATE)` 인 게시글이다.
- `status=BLOCKED` , `PRIVATE` , `DRAFT` , 삭제 게시글은 제외한다.
- `ALL` 기본 정렬은 `publishedAt DESC` 다.
- `FOLLOWING` 은 인증을 필수로 한다.
- 사용자 차단 관계가 있으면 서로의 게시글을 결과에서 제외한다.

- 카드의 동행 유형은 단일 `companionType` 으로 반환한다.
- 카드 작성자 정보에 `birdType` 을 포함한다.

입력 / Request DTO

- `tab: ALL | POPULAR | FOLLOWING`
- `period: WEEK | MONTH | SEASON | YEAR?`
- `cursor` , `size`

출력 / Response DTO

- `postId` , `thumbnailUrl` , `title`
- `author{userId, nickname, birdType}`
- `region` , `companionType` , `themes`
- `viewCount` , `saveCount` , `shareCount` , `savedRoute`
- `nextCursor`

예외사항 및 검증 로직

- `FOLLOWING` 비로그인 접근은 `401 UNAUTHORIZED` 다.

연동 API 엔드포인트

- `GET /api/community/posts`

3.9.2 기능명: 인기 기간 필터

기능 설명

- `WEEK` : 최근 7일
- `MONTH` : 최근 30일이며 기본값
- `SEASON` : 최근 3개월
- `YEAR` : 해당 연도 1월 1일부터 현재까지

세부 로직 및 상태 변화

- 기간별 이벤트를 `post_daily_metrics` 또는 원본 히스토리 테이블에서 합산한다.
- 인기 점수는 `조회수 × 1 + 저장수 × 3 + 공유수 × 5` 다.
- 동점은 최신 게시글 순으로 정렬한다.
- Redis 캐시는 사용하지 않고 MVP에서는 DB 집계 또는 정기 배치 결과를 사용한다.

연동 API 엔드포인트

- `GET /api/community/posts?tab=POPULAR&period=MONTH`

3.9.3 기능명: 커뮤니티 검색

기능 설명

- 공개 가능한 커뮤니티 게시글의 지역, 태그, 제목과 본문을 검색한다.
- 초성 검색, 오타 보정과 자동완성은 MVP에서 제외한다.

세부 로직 및 상태 변화

- `status=PUBLISHED`, `visibility IN (PUBLIC, MEMO_PRIVATE)` 인 게시글만 검색한다.
- 차단·삭제·비공개·임시 저장 게시글은 제외한다.
- 검색 가중치는 다음 순서다.
 1. 지역명 정확히 일치
 2. 태그 정확히 일치
 3. 제목에 검색어 포함
 4. 본문에 검색어 포함
- 같은 우선순위에서는 `createdAt DESC` 로 정렬한다.
- MVP 검색 구현은 MySQL `LIKE` 방식으로 통일하며 Full-Text 인덱스는 사용하지 않는다.
- 지역·태그 정확 일치는 `=` 조건으로 판정하고 제목·본문 포함 검색은 이스케이프 처리한 `LIKE` 조건으로 구현한다.

입력 / Request DTO

- `query: String`

- `regionCode: String?`
- `theme: String?`
- `cursor` , `size`

예외사항 및 검증 로직

- 지나치게 긴 검색어는 `400 SEARCH_QUERY_TOO_LONG` 이다.

연동 API 엔드포인트

- `GET /api/community/posts/search`

3.9.4 기능명: 조회수 증가

기능 설명

- 로그인 사용자의 게시글 상세 조회를 사용자당 게시글별 24시간에 한 번만 집계한다.

세부 로직 및 상태 변화

- `post_view_histories` 에 `postId` , `viewerUserId` , `viewedAt` 을 기록한다.
- 동일 사용자의 동일 게시글 조회가 최근 24시간 이내 존재하면 증가시키지 않는다.
- 작성자 본인의 조회는 집계하지 않는다.
- 비로그인 조회는 집계하지 않는다.
- 유효한 최초 조회일 때만 게시글 조회수와 일별 통계를 증가시킨다.
- 사용자 차단 관계 또는 접근 불가 게시글은 집계하지 않는다.

입력 / Request DTO

- Request Body 없음. JWT 사용자 ID를 사용한다.

출력 / Response DTO

- 성공 또는 중복 조회 모두 `204 No Content`

예외사항 및 검증 로직

- 비로그인 호출은 조회수 증가 없이 204 로 처리하거나 엔드포인트 자체를 호출하지 않도록 한다.
- 접근 불가능한 게시글은 404 POST_NOT_FOUND 다.

연동 API 엔드포인트

- POST /api/posts/{postId}/views
-

3.9.5 기능명: 공유수 증가

기능 설명

- 실제 공유 완료 여부와 관계없이 공유 버튼 클릭 수를 집계한다.

세부 로직 및 상태 변화

- 공유 버튼 클릭 시 shareCount 와 일별 통계를 증가시킨다.
- PUBLIC , MEMO_PRIVATE 게시글만 공유할 수 있다.
- 채널을 선택적으로 기록한다.

입력 / Request DTO

- channel: KAKAO | LINK | OTHER

예외사항 및 검증 로직

- 비공개·차단 게시글은 공유할 수 없다.
- 잘못된 채널은 400 INVALID_SHARE_CHANNEL 이다.

연동 API 엔드포인트

- POST /api/posts/{postId}/shares
-

3.9.6 기능명: 게시글 신고

기능 설명

- 로그인 사용자는 운영정책 위반 게시글을 신고할 수 있다.

- 신고 내역은 `reports` 테이블에 저장한다.
- 관리자 화면과 관리자 API는 제공하지 않는다.

세부 로직 및 상태 변화

- JWT 사용자 ID를 신고자로 저장한다.
- 신고 대상 Post 존재 여부와 접근 가능 여부를 검증한다.
- 신고 접수만으로 게시글을 자동 차단하지 않는다.
- 운영자가 DB에서 `posts.status=BLOCKED` 로 변경하면 일반 조회에서 즉시 제외한다.
- 동일 사용자는 동일 게시글을 사유와 관계없이 영구적으로 한 번만 신고할 수 있다.
- `reports` 테이블에 `(reporter_user_id, post_id)` 유니크 제약을 적용한다.
- 영구 1회 제한을 유지하기 위해 신고 레코드는 게시글 차단·삭제 여부와 관계없이 운영 이력으로 보관한다.

입력 / Request DTO

- `postId: Long`
- `reasonCode: SPAM | ABUSE | INAPPROPRIATE | OTHER`
- `description: String?`

출력 / Response DTO

- `reportId: Long`
- `status: RECEIVED`
- `createdAt: LocalDateTime`

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED` 다.
- 존재하지 않는 게시글은 `404 POST_NOT_FOUND` 다.
- 중복 신고는 `409 REPORT_ALREADY_SUBMITTED` 로 처리한다.

연동 API 엔드포인트

- `POST /api/reports`

3.10 경로 저장

3.10.1 기능명: 경로 저장·취소

기능 설명

- 로그인 사용자는 다른 사용자의 `PUBLIC`, `MEMO_PRIVATE` 여행 게시글 경로를 저장할 수 있다.
- 로그인 사용자는 자신이 생성한 AI 결과 미리보기에서 `경로 저장` 버튼을 눌러 해당 경로를 영구 보관할 수 있다.
- 자신의 여행 게시글은 저장할 수 없지만, 자신에게 발급된 AI 미리보기는 저장할 수 있다.
- 저장한 경로는 실제 여행 일정인 내 여행과 구분하여 관리한다.
- 저장한 경로는 경로 데이터를 복사한 별도 스냅샷이 아니라 원본 참조 데이터와의 연결 관계만 유지한다.
- AI 미리보기를 경로로 저장한 이후에도 소유자는 제목, 설명, 해시태그와 Day별 경로 구성을 계속 수정할 수 있다.
- Post 기반 `SavedRoute`는 `SavedRoute → Post → Trip` 원본 참조 관계를 유지하되, Post 발행 후 Trip 경로 잠금으로 저장한 경로가 예고 없이 바뀌지 않도록 한다.

세부 로직 및 상태 변화

- `SavedRoute` 는 `sourceType` 과 `sourceId` 로 원본을 참조한다.
 - 게시글 경로: `sourceType=POST`, `sourceId=postId`
 - AI 미리보기 경로: `sourceType=AI_PREVIEW`, `sourceId=previewId`
- 저장 관계에는 사용자 ID, 원본 유형, 원본 ID, 저장일과 `sourceAvailable` 을 저장한다.
- 게시글 경로 저장 시 Post의 제목·장소·순서·이미지 등의 데이터를 `SavedRoute` 에 복사하지 않는다.
- Post의 제목, 본문, 대표 이미지, 첨부 이미지, 해시태그와 공개 범위 등 허용된 비경로 수정은 저장 경로 조회에도 즉시 반영된다.
- Post가 존재하는 동안 연결 Trip의 지역·기간·Day·장소·방문 순서는 잠겨 있으므로 Post 기반 `SavedRoute`의 경로 구조는 변경되지 않는다.

- 원본 Post가 삭제·비공개·차단되면 저장 관계 행은 즉시 삭제하지 않고 `sourceAvailable=false` 로 변경한다.
- `sourceAvailable=false` 인 저장 관계는 일반 사용자의 저장 경로 목록에서 노출하지 않으며, 원본 제목·경로·이미지 등의 실제 데이터도 반환하지 않는다.
- AI 미리보기는 생성 시 기본적으로 `retentionStatus=TEMPORARY` , `expiresAt=생성시각+24시간` 으로 저장한다.
- 사용자가 AI 결과 화면에서 `경로 저장` 을 누르면 다음 작업을 하나의 트랜잭션으로 처리한다.
 1. `previewId` 가 현재 사용자의 미리보기인지 확인한다.
 2. 미리보기가 아직 만료되지 않았는지 확인한다.
 3. 해당 미리보기의 `retentionStatus` 를 `PERMANENT` 로 변경한다.
 4. `expiresAt=null` , `savedAt=현재시각` 으로 변경한다.
 5. `sourceType=AI_PREVIEW` , `sourceId=previewId` 인 `SavedRoute` 관계를 생성한다.
- 경로 저장된 AI 미리보기는 24시간 만료 및 자동 삭제 대상에서 제외하고 사용자의 저장 공간에 영구 보관한다.
- AI 경로를 저장할 때 별도의 미리보기 복사본이나 스냅샷을 만들지 않고 동일한 `previewId` 를 영구 원본으로 전환한다.
- 경로 저장은 AI 미리보기를 읽기 전용으로 잠그는 동작이 아니다.
- 저장 후 사용자가 AI 미리보기를 수정하면 동일한 Preview 원본을 갱신하며 `SavedRoute` 에 즉시 반영된다.
- 저장 후 수정 시 최초 `savedAt` 은 유지하고 `updatedAt` 과 `version` 을 갱신한다.
- `jobId` 는 AI 작업 상태를 식별하는 값이므로 경로 저장 원본으로 사용하지 않는다.
- 사용자가 `내 여행에 답기` 를 누르면 미리보기 내용을 기반으로 `Trip` , `TripDay` , `TripPlace` 를 생성한다.
- 이미 생성된 Trip은 Preview와 별도 객체이므로 이후 저장된 AI 경로를 수정해도 기존 Trip에는 자동 반영하지 않는다.
- `내 여행에 답기` 와 `경로 저장` 은 서로 독립된 기능이다.
- 동일한 원본 경로의 중복 저장 요청은 멍등하게 처리한다.
- 게시글 경로 저장·취소 시 `saveCount` 갱신을 하나의 트랜잭션으로 처리한다.

- AI 미리보기 경로는 커뮤니티 게시글이 아니므로 Post의 `saveCount`에는 포함하지 않는다.
- AI 미리보기 경로 저장 취소 후 해당 `previewId`를 참조하는 활성 SavedRoute 수가 0개가 되면 Preview를 `PERMANENT`에서 `TEMPORARY`로 강등한다.
- 강등 시 `savedAt=null`, `expiresAt=저장 취소 시각+24시간`으로 재설정하며, 이후 24시간 만료 배치가 일반 TEMPORARY Preview와 동일하게 콘텐츠를 삭제한다.
- Preview를 기반으로 이미 생성된 Trip은 별도 객체이므로 Preview 강등·만료의 영향을 받지 않는다.
- 만료 전에 사용자가 다시 경로 저장하면 동일 Preview를 다시 `PERMANENT`로 전환한다.

입력 / Request DTO

- 게시글 경로 저장·취소: Path `postId: Long`
- AI 미리보기 경로 저장·취소: Path `previewId: Long`
- 별도의 Request Body는 없다.
- AI 미리보기 내용 수정 요청은 3.19.6에서 처리한다.

출력 / Response DTO

- 저장·취소 성공 시 `204 No Content`를 반환한다.

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED`다.
- 존재하지 않는 Post는 `404 POST_NOT_FOUND`다.
- 비공개·차단·삭제 Post는 저장할 수 없다.
- 자신의 Post를 저장하면 `400 CANNOT_SAVE_OWN_ROUTE`를 반환한다.
- 존재하지 않는 AI 미리보기는 `404 AI_PREVIEW_NOT_FOUND`다.
- 다른 사용자의 AI 미리보기이면 `403 AI_PREVIEW_ACCESS_DENIED`다.
- 이미 만료된 미리보기이면 `410 AI_PREVIEW_EXPIRED`다.
- 중복 저장·취소는 멍등하게 처리한다.

연동 API 엔드포인트

- `PUT /api/users/me/saved-routes/posts/{postId}`
- `DELETE /api/users/me/saved-routes/posts/{postId}`
- `PUT /api/users/me/saved-routes/ai-previews/{previewId}`
- `DELETE /api/users/me/saved-routes/ai-previews/{previewId}`

3.10.2 기능명: 저장한 경로 목록 조회

기능 설명

- 사용자가 저장한 게시글 경로와 영구 저장한 AI 미리보기 경로를 저장일 내림차순으로 조회한다.
- 게시글 경로와 AI 미리보기 경로는 동일한 저장 경로 목록에서 `sourceType` 으로 구분한다.
- 저장된 AI 경로는 저장 후 수정 가능한 원본이므로 목록과 상세에서 항상 현재 최신 Preview 내용을 보여준다.
- 게시글 경로는 발행 후 잠긴 Trip의 경로를 조회하므로 저장 시점 이후 경로 구조가 예고 없이 바뀌지 않는다.

세부 로직 및 상태 변화

- `SavedRoute` 의 원본 참조를 기준으로 최신 데이터를 조회한다.
- `sourceType=POST` 이면 현재 Post와 연결된 Trip·장소·이미지 정보를 조회한다.
- `sourceType=AI_PREVIEW` 이면 영구 보관 상태의 AI 미리보기와 PreviewDay·PreviewPlace 정보를 조회한다.
- 영구 저장된 AI 미리보기는 `expiresAt` 이 없으며 계속 조회·수정할 수 있다.
- AI 미리보기의 제목, 설명, 해시태그, Day별 장소와 순서가 변경되면 저장 경로 조회 응답에도 즉시 반영한다.
- Post 기반 저장 경로에는 Post의 허용된 비경로 수정만 즉시 반영되며, 연결 Trip의 경로 구조는 Post 삭제 전까지 수정할 수 없다.
- 원본이 삭제·비공개·차단되어 `sourceAvailable=false` 가 된 관계는 DB에는 유지하지만 일반 사용자 목록 결과에서는 제외한다.
- 원본 게시글이 삭제되면 타 사용자의 북마크 화면에서 해당 항목과 실제 원본 데이터가 보이지 않는다.

- 같은 Trip에서 새 Post가 발행되어도 과거 삭제 Post의 저장 관계를 새 Post에 연결하지 않는다.
- 커서 페이지네이션을 사용한다.

입력 / Request DTO

- Query `cursor: Long?`
- Query `size: Int = 20`

출력 / Response DTO

- `items`
 - `savedRouteId: Long`
 - `sourceType: POST | AI_PREVIEW`
 - `sourceId: Long`
 - `sourceAvailable: Boolean`
 - `title: String`
 - `thumbnailUrl: String?`
 - `author: AuthorSummary?`
 - `region`
 - `savedAt: LocalDateTime`
 - `updatedAt: LocalDateTime`
 - `editable: Boolean`
- `nextCursor: Long?`
- 일반 사용자 목록에는 `sourceAvailable=true` 인 항목만 반환한다.
- `sourceType=AI_PREVIEW` 이고 현재 사용자가 소유자이면 `editable=true` 를 반환한다.
- 다른 사용자의 Post를 참조하는 저장 경로는 `editable=false` 를 반환한다.

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED` 다.

- 영구 저장된 AI 원본 데이터가 비정상적으로 존재하지 않으면 해당 저장 관계를 `sourceAvailable=false` 로 변경하고 목록에서 제외한다.

연동 API 엔드포인트

- `GET /api/users/me/saved-routes`
-

3.11 포토맵

3.11.1 기능명: 전국 포토맵과 방문 지역 수 조회

기능 설명

- 사용자가 발행한 게시글의 장소를 기준으로 포토맵을 생성한다.
- 일정에만 등록되고 게시글로 발행하지 않은 장소는 방문으로 인정하지 않는다.

세부 로직 및 상태 변화

- 삭제되지 않은 `PUBLISHED` Post와 `post_places` 를 기준으로 집계한다.
- `PRIVATE` 게시글도 작성자 본인의 포토맵에는 포함한다.
- DRAFT, BLOCKED, 삭제 Post는 제외한다.
- 동일 장소가 여러 게시글에 있어도 `COUNT(DISTINCT placeId)` 로 한 곳으로 집계한다.
- 방문 지역 단위는 광역시·도다.
- 데이터 증가로 성능이 저하되면 사용자·지역별 집계 테이블을 분리한다.

출력 / Response DTO

- `regions[{regionCode, regionName, visitedPlaceCount, recordCount}]`
- `totalVisitedRegionCount`

연동 API 엔드포인트

- `GET /api/users/me/photomap/regions`
-

3.11.2 기능명: 지역별 방문 장소 조회

기능 설명

- 광역시·도별 방문 장소와 연결 게시글을 조회한다.
- 동일 장소에 여러 게시글이 있으면 가장 최근 게시글로 기본 이동한다.
- 포토맵 리스트 화면에서는 해당 장소의 나머지 게시글도 확인할 수 있다.

세부 로직 및 상태 변화

- 장소별 가장 최근 `publishedAt` 의 Post를 `representativePostId` 로 반환한다.
- 같은 장소의 다른 Post ID 목록을 최신순으로 반환한다.
- 삭제·DRAFT·BLOCKED 게시글은 제외하고 작성자 본인의 PRIVATE 게시글은 포함한다.

출력 / Response DTO

- `placeId` , `name` , `latitude` , `longitude`
- `visitCount`
- `representativePostId`
- `postIds`

연동 API 엔드포인트

- `GET /api/users/me/photomap/regions/{regionCode}/places`
-

3.12 홈

3.12.1 기능명: 홈 데이터 조회

기능 설명

- 홈에는 오늘의 추천 장소 7개, 축제·이벤트, 추천 기록과 날씨 헤드라인을 표시한다.
- 추천 장소는 전체 사용자에게 공통으로 제공한다.

세부 로직 및 상태 변화

- MVP 추천 장소 7개는 운영자가 DB에서 직접 지정한 고정 장소다.

- 추천 기록은 커뮤니티 인기 점수와 랜덤 셔플을 혼합한다.
- 대상 게시글은 `PUBLISHED`, `PUBLIC` 또는 `MEMO_PRIVATE`이며 `BLOCKED`·삭제 게시글은 제외한다.
- 날씨 기준 위치는 사용자의 현재 위치다.
- 위치 권한을 거부하거나 좌표가 없으면 서울을 기본 도시로 사용한다.
- `[추가 확정 필요]` 실제 날씨 API 공급자와 공급자 장애 시 마지막 성공 데이터·고정 문구·빈 날씨 중 어떤 대체 응답을 사용할지는 제공된 결정사항에 포함되어 있지 않다.
- 축제 데이터는 한국관광공사 TourAPI 국문 관광정보 API에서 매일 새벽 1회 수집·갱신한다.
- 한 섹션 실패로 홈 전체가 실패하지 않도록 부분 응답을 허용한다.
- Redis는 사용하지 않고 DB와 필요 시 애플리케이션 단기 캐시를 사용한다.

입력 / Request DTO

- `latitude: BigDecimal?`
- `longitude: BigDecimal?`

출력 / Response DTO

- `headline{season, weatherType, text, baseLocation}`
- `recommendedPlaces`
- `monthlyEvents`
- `recommendedPosts`

연동 API 엔드포인트

- `GET /api/home`

3.13 이웃새 및 사용자 차단

3.13.1 기능명: 팔로우·언팔로우

기능 설명

- 상호 승인 없는 단방향 팔로우 구조를 사용한다.
- 언팔로우 시 관계 데이터를 삭제한다.
- 비공개 계정과 팔로우 승인 요청은 MVP에서 제공하지 않는다.

세부 로직 및 상태 변화

- `(followerUserId, followingUserId)` 관계를 생성한다.
- 자기 자신 팔로우를 차단한다.
- 중복 팔로우·언팔로우는 먹등 처리한다.
- 차단 관계가 존재하면 팔로우할 수 없다.

연동 API 엔드포인트

- `PUT /api/users/{targetUserId}/follow`
- `DELETE /api/users/{targetUserId}/follow`

3.13.2 기능명: 팔로워·팔로잉 목록 조회

기능 설명

- 마이페이지와 타인 프로필에서 팔로워 수와 팔로잉 수를 구분한다.
- 각각 클릭하면 해당 사용자 목록을 조회한다.

입력 / Request DTO

- `type: FOLLOWERS | FOLLOWINGS`
- `cursor`, `size`

출력 / Response DTO

- `userId`, `nickname`, `birdType`, `trait`, `followedAt`
- `nextCursor`

연동 API 엔드포인트

- `GET /api/users/{userId}/follows?type=FOLLOWERS`

- `GET /api/users/{userId}/follows?type=FOLLOWINGS`

3.13.3 기능명: 사용자 차단·해제·목록 조회

기능 설명

- 로그인 사용자는 특정 사용자를 차단하거나 차단 해제할 수 있다.
- 앱 마켓 심사 대응을 위해 MVP에 포함한다.

세부 로직 및 상태 변화

- `(blockerUserId, blockedUserId)` 관계를 저장한다.
- 자기 자신 차단은 허용하지 않는다.
- 차단 시 두 사용자 사이의 팔로우 관계를 제거한다.
- 차단 관계가 어느 방향으로든 존재하면 서로의 게시물, 프로필과 팔로우 목록 노출을 제한한다.
- 차단 사용자 사이의 팔로우·경로 저장 등 신규 상호작용을 차단한다.
- 기존에 저장한 상대방 경로는 `sourceAvailable=false`로 처리하여 원본 내용을 숨긴다.

입력 / Request DTO

- Path `targetUserId: Long`

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED` 다.
- 자기 자신 차단은 `400 CANNOT_BLOCK_SELF` 다.
- 존재하지 않는 사용자는 `404 USER_NOT_FOUND` 다.
- 중복 차단·해제는 멍등 처리한다.

연동 API 엔드포인트

- `PUT /api/users/{targetUserId}/block`
- `DELETE /api/users/{targetUserId}/block`
- `GET /api/users/me/blocked-users`

3.14 마이페이지

3.14.1 기능명: 마이페이지 집계 조회

기능 설명

- 프로필, 파트너 새, 기록 수, 방문 지역 수, 팔로잉 수를 조회한다.
- 내 여행, 캘린더, 저장 경로와 저장 장소 메뉴를 제공한다.
- 리워드 포인트 메뉴는 MVP에서 제외한다.

세부 로직 및 상태 변화

- 기록 수는 삭제되지 않은 Post 중 DRAFT를 제외하여 집계한다.
- PRIVATE 게시글은 기록 수에 포함한다.
- BLOCKED 게시글은 작성자 본인의 `postCount` 에서도 제외한다.
- 방문 지역 수는 포토맵과 동일하게 광역시·도 기준이다.
- 대표 이웃새 수치는 사용자가 팔로우한 `followingCount` 다.
- 팔로워 수와 팔로잉 수를 각각 반환한다.
- 파트너 새는 `birdType` 만 기준으로 프론트 이미지와 매핑한다.

출력 / Response DTO

- `profile{userId, nickname, introduction, birdType}`
- `statistics.postCount`
- `statistics.visitedRegionCount`
- `statistics.followerCount`
- `statistics.followingCount`

연동 API 엔드포인트

- `GET /api/users/me/mypage`
-

3.15 리워드 포인트

- 리워드 포인트 잔액, 적립, 사용, 거래내역과 관련 API는 MVP에서 완전히 제외한다.
 - `reward_accounts`, `reward_transactions` 테이블과 `/api/users/me/rewards`, `/api/users/me/reward-transactions` API를 구현하지 않는다.
-

3.16 통합 검색

3.16.1 기능명: 장소·기록·지역 통합 검색

기능 설명

- 로그인 사용자는 홈에서 장소, 여행 기록과 지역을 통합 검색한다.
- 결과 노출 우선순위는 장소 → 기록 → 지역이다.
- 각 타입별 최대 5개를 반환한다.
- 사용자 계정 검색은 포함하지 않는다.
- 최근 검색어 저장과 인기 검색어 집계는 MVP에서 제외한다.

세부 로직 및 상태 변화

- 장소 검색과 공개 게시글 검색을 조합한다.
- 게시글은 `PUBLISHED`, `PUBLIC` 또는 `MEMO_PRIVATE` 만 포함하고 `BLOCKED`·삭제 게시글을 제외한다.
- 장소 API 일일 한도가 초과되면 장소 섹션에 오류를 표시하고 기록·지역 결과만 부분 반환할 수 있다.
- 초성 검색, 오타 보정과 자동완성은 제공하지 않는다.

입력 / Request DTO

- `query: String`
- `limitPerType: Int = 5` 이며 최대 5
- `regionCode: String?`

출력 / Response DTO

- 응답 표시 순서: `places`, `posts`, `regions`

- 각 목록 최대 5개
- `hasMorePlaces` , `hasMorePosts` , `hasMoreRegions`

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED` 다.
- 빈 검색어는 `400 SEARCH_QUERY_REQUIRED` 다.

연동 API 엔드포인트

- `GET /api/search`

3.17 축제·이벤트

3.17.1 기능명: 진행·예정 축제 목록 및 상세 조회

기능 설명

- 홈과 전체보기 화면에 진행 중이거나 예정된 국내 축제를 표시한다.
- 종료된 축제는 MVP 목록과 과거 기록에서 노출하지 않는다.
- 축제 저장·찜 기능은 MVP에서 제외하고 프론트 하트 버튼도 제거한다.

세부 로직 및 상태 변화

- 한국관광공사 TourAPI 국문 관광정보 API에서 데이터를 수집한다.
- 매일 새벽 1회 배치 스케줄러로 내부 DB에 동기화한다.
- `endDate < 오늘` 인 축제는 조회에서 제외한다.
- 상태는 `UPCOMING` , `ONGOING` 만 반환한다.

입력 / Request DTO

- `from: LocalDate`
- `to: LocalDate`
- `regionCode: String?`
- `cursor` , `size`

출력 / Response DTO

- `eventId`, `name`, `region`, `placeName`
- `startDate`, `endDate`, `thumbnailUrl`
- `status: UPCOMING | ONGOING`

연동 API 엔드포인트

- `GET /api/events`
 - `GET /api/events/{eventId}`
-

3.18 지도·경로 표시

3.18.1 기능명: Day별 이동 경로 조회

기능 설명

- 선택 Day의 장소를 방문 순서 번호 핀과 단순 직선 점선으로 연결한다.
- 이동 수단 선택, 예상 이동시간과 총 거리는 제공하지 않는다.
- 네이버 Directions API는 MVP에서 호출하지 않는다.

세부 로직 및 상태 변화

- Trip의 Day 장소 좌표를 `order` 순으로 반환한다.
- 프론트엔드는 인접 좌표를 직선 점선으로 연결한다.
- 별도 경로 선 데이터를 DB에 저장하지 않는다.
- Trip 공개 범위에 따라 접근 권한을 검증한다.
 - 소유자: 전체 접근
 - `PUBLIC`, `MEMO_PRIVATE`: 로그인·비로그인 접근 허용
 - `PRIVATE`: 소유자만 허용
- `jobId`, `previewId` 는 본 API에서 사용하지 않는다.

입력 / Request DTO

- Path `tripId: Long`, `dayNumber: Int`
- 이동 수단 Query Parameter는 받지 않는다.

출력 / Response DTO

- `tripId`, `dayNumber`
- `places[{tripPlaceId, order, placeId, latitude, longitude}]`
- `routePoints[{latitude, longitude}]`
- `totalDistanceMeters`, `estimatedDurationMinutes` 는 반환하지 않는다.

예외사항 및 검증 로직

- 존재하지 않는 일정은 `404 TRIP_NOT_FOUND` 다.
- PRIVATE 비소유자 접근은 `404 TRIP_NOT_FOUND` 다.
- Day가 없으면 `404 TRIP_DAY_NOT_FOUND` 다.
- 장소가 0~1개면 좌표만 정상 반환한다.

연동 API 엔드포인트

- `GET /api/trips/{tripId}/days/{dayNumber}/route`

3.19 AI 여행 추천

3.19.1 기능명: 일반 조건 기반 AI 일정 추천 요청

기능 설명

- 사용자는 지역, 날짜, 단일 동행 유형, 테마와 일정 선호도로 AI 추천을 요청한다.
- AI-데이터 파트는 사전 수집·정규화한 관광지 데이터에서 후보 장소를 선정한다.
- 추천 요청 시 관광 API를 실시간 호출하지 않는다.
- 결과는 Trip에 즉시 저장하지 않고 AI 일정 미리보기로 제공한다.
- 생성된 미리보기는 기본적으로 24시간 동안 유효하지만, 사용자가 `경로 저장` 을 실행하면 영구 보관 상태로 전환된다.

세부 로직 및 상태 변화

- 클라이언트는 Spring Boot만 호출한다.
- 사용자별 AI 추천은 하루 최대 3회다.
- 일일 기준은 `Asia/Seoul` 00:00~23:59로 집계한다.
- 백엔드는 AI job을 `QUEUED` 로 생성하고 `jobId` 를 반환한다.
- 백엔드의 내부 비동기 디스패처는 AI 서버에 HTTP REST 요청을 전달하고 작업 접수 응답만 확인한 뒤 연결을 종료한다.
- AI 서버가 작업을 접수하면 job을 `PROCESSING` 으로 변경한다.
- AI 서버는 생성 완료 후 `POST /internal/ai-callbacks/trip-recommendations` 로 결과를 전달한다.
- Callback 결과 검증과 preview 저장이 완료되면 `SUCCEEDED` 로 변경한다.
- 실패하면 `FAILED` 로 변경한다.
- 미리보기 생성 시 `retentionStatus=TEMPORARY` , `expiresAt=생성시각+24시간` 을 설정한다.
- 사용자가 아무 저장 동작도 하지 않은 미리보기는 24시간 후 `EXPIRED` 로 변경하고 접근을 차단한다.
- 사용자가 `경로 저장` 을 누르면 동일한 미리보기를 `PERMANENT` 로 전환하고 `expiresAt` 을 제거하여 영구 보관한다.
- 사용자가 `내 여행에 담기` 를 누르면 미리보기 내용을 기반으로 실제 Trip을 생성한다.
- 클라이언트는 폴링으로 상태를 확인한다.
- 진행 중 작업 취소는 지원하지 않는다.

입력 / Request DTO

- `regionCode` , `startDate` , `endDate`
- `companionType`
- `themes`
- `pace`
- `requestType: GENERAL`

출력 / Response DTO

- 202 Accepted
- jobId
- status: QUEUED
- requestedAt
- statusUrl

예외사항 및 검증 로직

- 비로그인 요청은 401 UNAUTHORIZED 다.
- 일일 3회 초과는 429 AI_DAILY_REQUEST_LIMIT_EXCEEDED 다.
- 잘못된 날짜·지역·동행·테마·선호도는 각각 400 오류로 처리한다.
- AI 서버에 작업 자체를 전달하지 못하면 503 AI_SERVICE_UNAVAILABLE 이다.
- AI 작업 접수 이후 Callback이 제한시간 안에 도착하지 않으면 job을 FAILED 로 변경하고 AI_CALLBACK_TIMEOUT 을 기록한다.

연동 API 엔드포인트

- POST /api/ai/trip-recommendations

3.19.2 기능명: 저장 장소 기반 AI 일정 추천 요청

기능 설명

- 사용자는 자신이 저장한 장소를 선택하여 AI 추천 우선순위에 반영한다.
- 선택 장소는 일정 필수 포함이 아니라 가중치 상향 기준이다.
- AI는 선택하지 않은 새로운 장소를 추가 추천할 수 있다.

세부 로직 및 상태 변화

- 선택한 placeId 가 요청 사용자의 저장 장소인지 검증한다.
- 선택 가능한 장소는 요청 regionCode 와 동일 지역으로 제한한다.
- 최소 선택 장소 수는 1개다.
- 선택 장소 좌표·카테고리·지역정보를 AI에 전달한다.

- AI-데이터 파트가 정규화 데이터에서 후보 장소를 선정한다.
- 결과는 preview로 저장한다.

입력 / Request DTO

- 일반 조건
- `requestType: SAVED_PLACES`
- `savedPlaceIds: List<Long>` 최소 1개

예외사항 및 검증 로직

- 저장하지 않은 장소는 `403 SAVED_PLACE_ACCESS_DENIED` 다.
- 선택 장소가 없으면 `422 INSUFFICIENT_SAVED_PLACES` 다.
- 다른 지역 장소는 `422 INCOMPATIBLE_PLACE_REGIONS` 다.

연동 API 엔드포인트

- `POST /api/ai/trip-recommendations`

3.19.3 기능명: 일정 위시리스트 기반 AI 경로 배치 요청

기능 설명

- 특정 Trip의 위시리스트 장소를 Day별 경로로 배치한다.
- 위시리스트 장소는 모두 일정에 반드시 포함한다.
- AI는 필요한 보완 장소를 추가 추천할 수 있다.
- 결과는 기존 Trip에 즉시 반영하지 않고 preview로 제공한다.
- 발행된 Post가 연결되어 경로가 잠긴 Trip에는 AI 경로 배치 요청 자체를 허용하지 않는다.

세부 로직 및 상태 변화

- Trip 소유권과 수정 가능 여부를 검증한다.
- `cancelledAt` 이 존재하는 Trip은 AI 경로 배치 요청을 `409 TRIP_CANCELLED_READ_ONLY` 로 차단한다.

- AI 비용 발생 전에 연결된 발행 Post 존재 여부를 먼저 검증한다.
- 위시리스트가 비어 있으면 요청을 차단한다.
- 위시리스트와 기존 일정 장소를 내부 `placeId` 로 전달한다.
- `existingPlaceMode=KEEP` 이면 기존 장소 자체는 유지하지만 기존 Day와 순서는 고정하지 않고 AI가 재배치한다.
- `existingPlaceMode=REPLACE` 이면 기존 일정 장소를 필수 유지하지 않는다.
- 위시리스트 장소는 반드시 결과에 포함한다.
- AI는 보완 장소를 추가할 수 있다.
- preview 생성 시 기존 Trip을 변경하지 않고 사용자가 확정할 때만 반영한다.
- 미리보기 적용 시점에도 경로 잠금을 다시 검증하여, 요청 이후 Post가 발행된 경쟁 상황을 차단한다.

입력 / Request DTO

- Path `tripId`
- `requestType: TRIP_WISHLIST`
- `existingPlaceMode: KEEP | REPLACE`

예외사항 및 검증 로직

- 빈 위시리스트는 `400 EMPTY_WISHLIST` 다.
- 비소유자는 `403 TRIP_ACCESS_DENIED` 다.
- 존재하지 않는 장소는 `404 PLACE_NOT_FOUND` 다.
- 발행 Post가 연결된 Trip은 `409 TRIP_ROUTE_LOCKED_BY_PUBLISHED_POST` 다.
- 취소된 Trip은 `409 TRIP_CANCELLED_READ_ONLY` 다.

연동 API 엔드포인트

- `POST /api/trips/{tripId}/ai-route-recommendations`

3.19.4 기능명: AI 작업 상태 조회

기능 설명

- 클라이언트는 `jobId` 로 상태를 폴링한다.
- 실시간 SSE, WebSocket과 Push 완료 알림은 MVP에서 제외한다.
- 진행 중 작업 취소는 지원하지 않는다.
- `jobId` 는 작업 상태 조회에만 사용하며 경로 저장의 원본 식별자로 사용하지 않는다.

세부 로직 및 상태 변화

- 상태는 `QUEUED` , `PROCESSING` , `SUCCEEDED` , `FAILED` , `EXPIRED` 다.
- `QUEUED` : 처리 대기
- `PROCESSING` : 후보 선정과 일정 생성 진행
- `SUCCEEDED` : 검증 및 preview 저장 완료
- `FAILED` : AI 처리·검증·저장 실패
- `EXPIRED` : 미저장 임시 미리보기의 유효기간이 끝나 상세 내용이 삭제된 상태
- 성공 상태에서만 사용 가능한 `previewId` 를 반환한다.
- `retentionPolicy=TEMPORARY` 인 미저장 미리보기만 24시간 만료 대상으로 한다.
- 사용자가 `경로 저장` 을 실행하여 `retentionPolicy=PERMANENT` 가 된 미리보기는 만료하지 않는다.
- 영구 저장된 미리보기와 연결된 AI 작업은 `SUCCEEDED` 상태를 유지한다.
- `TEMPORARY` 미리보기는 만료 전까지 수정할 수 있고 `PERMANENT` 미리보기는 저장 후에도 계속 수정할 수 있다.
- 미리보기 수정 자체는 현재 `expiresAt` 을 연장하지 않는다.
- 마지막 AI SavedRoute 취소로 `PERMANENT → TEMPORARY` 강등된 경우에만 `expiresAt` 을 취소 시각부터 24시간 후로 새로 설정한다.
- 강등 직후 연결 AI Job은 `SUCCEEDED` 를 유지하고, 새 `expiresAt` 이 지난 뒤 만료 배치가 실행되면 `EXPIRED` 로 변경한다.
- 사용자가 `내 여행에 답기` 만 실행하고 경로 저장하지 않은 경우 Trip은 영구 유지되지만 임시 Preview는 24시간 후 만료된다.
- 만료 스케줄러는 `expiresAt <= now` , `retentionPolicy=TEMPORARY` , `SavedRoute 없음` 조건을 모두 충족한 Preview만 처리한다.
- 만료 처리 시 PreviewDay, PreviewPlace, 해시태그, 제목과 설명 등 미리보기 내용을 삭제한다.

- `previewId`, 소유자 ID, 생성·만료시각과 상태만 가진 최소 tombstone 행은 유지하여 기존 preview 접근에 `410 AI_PREVIEW_EXPIRED` 를 반환할 수 있게 한다.
- 연결 AI Job 행은 삭제하지 않고 `status=EXPIRED`, `expiredAt` 을 기록하여 MVP 기간 동안 계속 보관한다.
- AI Job에는 요청 시각, 요청 유형, 처리시간, 성공·실패·만료 상태와 오류코드 등 운영 분석에 필요한 최소 메타데이터를 유지한다.
- MVP에서는 EXPIRED AI Job의 자동 삭제 기간을 두지 않는다.

출력 / Response DTO

- `jobId`
- `status: QUEUED | PROCESSING | SUCCEEDED | FAILED | EXPIRED`
- `previewId: Long?`
- `previewAvailable: Boolean`
- `previewRetentionStatus: TEMPORARY | PERMANENT | EXPIRED`
- `editable: Boolean`
- `requestedAt`, `startedAt`, `completedAt`, `expiresAt`, `expiredAt`
- `savedAt: LocalDateTime?`
- `error{code, message, retryable}`
- `EXPIRED` 응답에서는 추적을 위해 `previewId` 를 유지할 수 있으나 `previewAvailable=false`, `editable=false` 를 반환한다.

예외사항 및 검증 로직

- 없는 작업은 `404 AI_JOB_NOT_FOUND` 다.
- 타인 작업은 `403 AI_JOB_ACCESS_DENIED` 다.
- 만료 작업은 상태 응답에서 `EXPIRED` 를 반환한다.
- 만료 Preview 상세 접근은 `410 AI_PREVIEW_EXPIRED` 로 처리한다.
- 영구 저장된 미리보기에는 `expiresAt=null`, `previewAvailable=true`, `editable=true` 를 반환한다.

연동 API 엔드포인트

- `GET /api/ai/trip-recommendations/{jobId}`

3.19.5 기능명: AI 결과 검증 및 일정 미리보기 생성

기능 설명

- AI는 일정 제목, 설명, Day별 내부 `placeId`, 순서, 추천 이유와 선택적인 시간·체류시간을 반환한다.
- `placeId`, `order`, `reason`은 필수다.
- 추천 시간과 체류시간은 선택값이다.
- AI 결과에 일부 오류가 있으면 전체 결과를 실패 처리한다.
- 검증된 결과는 실제 Trip이 아니라 24시간 유효한 임시 미리보기로 최초 생성한다.
- 생성된 Preview는 임시 또는 영구 보관 상태와 관계없이 소유자가 수정할 수 있는 경로 원본이다.

세부 로직 및 상태 변화

- AI-데이터 파트가 정규화 관광지 데이터에서 후보를 선정한다.
- 백엔드는 반환된 모든 `placeId`가 내부 DB에 존재하는지 검증한다.
- 후보 장소 선정 주체가 AI-데이터 파트이므로 백엔드가 별도 전달 후보목록 포함 여부를 필수 검증하지 않는다.
- Day 번호가 여행 기간 안에 있는지 검증한다.
- Day별 `order` 누락·중복을 검증한다.
- 동일 장소의 비정상 중복을 검증한다.
- `reason`은 공백 제외 최소 10자, 최대 100자다.
- `recommendedTime`은 유효한 시간 형식이어야 한다.
- `durationMinutes`가 있으면 0보다 커야 한다.
- HTML·Script를 이스케이프 처리한다.
- 검증 실패 시 preview를 생성하지 않고 job을 `FAILED`로 변경한다.
- 검증 성공 시 Preview, PreviewDay, PreviewPlace와 job 상태를 하나의 트랜잭션으로 저장한다.

- 신규 Preview는 `retentionStatus=TEMPORARY`, `expiresAt=createdAt+24시간`, `savedAt=null`로 저장한다.
- TEMPORARY Preview가 만료되면 Preview의 경로·텍스트 콘텐츠를 삭제하고 최소 tombstone과 AI Job 기록만 유지한다.
- 경로 저장 시 Preview·PreviewDay·PreviewPlace를 복사하지 않고 동일 Preview의 `retentionStatus=PERMANENT`, `expiresAt=null`, `savedAt=현재시각`으로 변경한다.
- 영구 저장된 Preview는 저장 경로의 원본 `Source Data` 역할을 하며 `SavedRoute`가 해당 `previewId`를 계속 참조한다.
- 이후 사용자가 저장된 AI 경로를 수정하면 동일 Preview·PreviewDay·PreviewPlace를 갱신하며 별도 복사본은 생성하지 않는다.
- 이 단계에서는 Trip을 생성하거나 수정하지 않는다.

입력 / AI Result DTO

- `tripTitle: String`
- `summary: String`
- `days`
 - `day: Int`
 - `places`
 - `placeId: Long`
 - `order: Int`
 - `reason: String`
 - `recommendedTime: LocalTime?`
 - `durationMinutes: Int?`
- `requestId: String?`와 `schemaVersion: String?`은 Optional 필드로 받으며 누락되어도 AI 결과를 거부하지 않는다.
- Callback 결과의 작업 식별에는 백엔드가 발급한 `jobId: Long`을 필수값으로 사용한다.

출력 / 처리 결과

- 검증 성공 시 `previewId`를 생성하고 AI 작업 상태를 `SUCCEEDED`로 변경한다.
- 미리보기의 기본 보관 상태는 `TEMPORARY`다.

- `expiresAt` 은 생성 시각부터 24시간 후다.
- 사용자가 경로 저장을 완료하면 동일 `previewId` 가 `PERMANENT` 상태로 전환된다.
- 영구 전환 이후에도 동일 `previewId` 를 대상으로 수정 API를 사용할 수 있다.

예외사항 및 검증 로직

- 구조 오류는 `INVALID_AI_RESPONSE` 다.
- 내부 장소 없음은 `AI_PLACE_NOT_FOUND` 다.
- `placeId` 누락은 `AI_PLACE_ID_REQUIRED` 다.
- 순서 오류는 `AI_PLACE_ORDER_INVALID` 다.
- 추천 이유 누락·길이 오류는 `AI_RECOMMENDATION_REASON_INVALID` 다.
- 시간 오류는 `AI_RECOMMENDED_TIME_INVALID` 다.
- 체류시간 오류는 `AI_DURATION_INVALID` 다.
- 일부 오류가 있어도 유효 부분만 저장하지 않고 전체를 `AI_RESULT_VALIDATION_FAILED` 로 처리한다.

내부 연동 방식

- 비동기 Callback 방식으로 확정한다.
- 백엔드는 AI 서버에 HTTP REST로 작업 요청을 전달하되 생성 결과 응답을 같은 연결에서 기다리지 않는다.
- AI 서버는 작업을 완료한 뒤 `POST /internal/ai-callbacks/trip-recommendations` 를 호출한다.
- Callback Request에는 `jobId` 와 AI Result DTO를 포함한다.
- `requestId` , `schemaVersion` 은 Optional이다.
- Callback은 동일 `jobId` 로 재전송되어도 Preview가 중복 생성되지 않도록 멍등 처리한다.
- Callback의 `jobId` 가 존재하지 않거나 이미 `FAILED` , `EXPIRED` 인 경우 결과를 저장하지 않는다.
- `[AI 제안·보안 기준]` 내부 Callback은 외부 사용자 API와 분리하고 내부 인증키 또는 요청 서명 검증을 적용한다.

내부 Callback API 응답

- 결과 수신 및 저장 성공: `200 OK`
 - `accepted: true`
 - `jobId: Long`
 - `previewId: Long`
 - `processedAt: LocalDateTime`
- 중복 Callback: 기존 처리 결과를 반환하는 멱등 `200 OK`
- 구조·검증 실패: `422 INVALID_AI_RESPONSE`
- 존재하지 않는 작업: `404 AI_JOB_NOT_FOUND`

3.19.6 기능명: AI 일정 미리보기 조회·수정

기능 설명

- 사용자는 자신에게 발급된 AI 일정 미리보기를 조회하고 내용을 수정할 수 있다.
- `TEMPORARY` 미리보기는 생성 후 24시간 내에만 조회·수정할 수 있다.
- `PERMANENT` 로 경로 저장된 미리보기는 24시간 제한 없이 계속 조회·수정할 수 있다.
- 경로 저장 이후에도 수정 가능하며, 저장은 현재 내용을 고정하거나 편집을 잠그는 기능이 아니다.
- 저장된 AI 경로를 수정하면 해당 경로를 참조하는 저장 목록과 상세 화면에 변경사항이 즉시 반영된다.

세부 로직 및 상태 변화

- JWT 사용자 ID와 Preview 소유자 ID를 비교하여 본인 미리보기인지 검증한다.
- 미리보기의 `retentionStatus` 와 `expiresAt` 을 확인한다.
- `TEMPORARY` 이면서 만료되지 않은 미리보기와 `PERMANENT` 미리보기만 수정할 수 있다.
- 사용자는 다음 항목을 수정할 수 있다.
 - 일정 제목 `tripTitle`
 - 일정 설명 `summary`
 - 해시태그 `hashtags`
 - Day별 장소 추가·삭제

- 장소의 Day 이동
- Day 안의 장소 방문 순서 변경
- 장소별 `recommendedTime`, `durationMinutes`
- 지역, 여행 시작일·종료일, 동행 유형, 테마와 일정 선호도처럼 AI 추천의 입력조건 자체를 변경하려면 기존 미리보기를 직접 수정하지 않고 새로운 AI 추천을 요청한다.
- 경로 전체를 수정하는 경우 클라이언트는 최종 Day와 장소 목록 전체를 전송하며 서버는 하나의 트랜잭션으로 반영한다.
- 장소 추가 시 내부 `placeId` 가 존재하는지 검증한다.
- 같은 장소는 동일 Day에 중복으로 포함할 수 없으며 다른 Day에는 포함할 수 있다.
- Day별 장소 수는 최대 15개다.
- 장소 삭제 시 해당 `PreviewPlace`에 연결된 사용자 수정 시간·체류시간 정보도 함께 삭제한다.
- Day별 `order` 는 1부터 연속되도록 재정렬한다.
- 수정 시 기존 `SavedRoute` 관계와 `previewId` 는 유지한다.
- 수정 완료 후 `updatedAt` 과 `version` 을 갱신하고, 최초 경로 저장 시각인 `savedAt` 은 변경하지 않는다.
- 수정된 Preview가 이미 `SavedRoute` 에 연결되어 있으면 별도의 저장 요청 없이 변경사항이 저장된 경로에 즉시 반영된다.
- Preview를 기반으로 이미 생성된 Trip은 별도 객체이므로 Preview 수정사항을 자동 반영하지 않는다. 변경된 경로로 Trip을 다시 만들거나 기존 Trip에 적용하려면 사용자의 별도 확정 동작이 필요하다.
- 동시 수정 충돌 방지를 위해 `version` 을 이용한 낙관적 락을 적용한다.

입력 / Request DTO

- 조회
 - Path `previewId: Long`
- 수정 DTO명은 `UpdateAiTripPreviewRequest` 다.
 - `tripTitle: String?`
 - `summary: String?`
 - `hashtags: List<String>?`

- `days: List<UpdateAiPreviewDayRequest>?`
 - `day: Int`
 - `places: List<UpdateAiPreviewPlaceRequest>`
 - `placeId: Long`
 - `order: Int`
 - `recommendedTime: LocalTime?`
 - `durationMinutes: Int?`
- `version: Long`
- `days` 가 전달되면 현재 경로의 최종 Day·장소 구성을 전체 목록으로 전달한다.

출력 / Response DTO

- `previewId: Long`
- `retentionStatus: TEMPORARY | PERMANENT`
- `editable: Boolean`
- `tripTitle: String`
- `summary: String`
- `hashtags: List<String>`
- `days: List<AiPreviewDayResponse>`
- `createdAt: LocalDateTime`
- `updatedAt: LocalDateTime`
- `expiresAt: LocalDateTime?`
- `savedAt: LocalDateTime?`
- `version: Long`

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED` 다.
- 존재하지 않는 미리보기는 `404 AI_PREVIEW_NOT_FOUND` 다.
- 다른 사용자의 미리보기는 `403 AI_PREVIEW_ACCESS_DENIED` 다.

- 만료된 임시 미리보기는 상세 콘텐츠가 삭제된 상태이므로 `410 AI_PREVIEW_EXPIRED` 다.
- 존재하지 않는 장소는 `404 PLACE_NOT_FOUND` 다.
- 동일 Day에 같은 장소가 중복되면 `409 AI_PREVIEW_PLACE_DUPLICATED` 다.
- Day별 장소 수가 15개를 초과하면 `422 AI_PREVIEW_DAY_PLACE_LIMIT_EXCEEDED` 다.
- Day 번호나 장소 순서가 올바르지 않으면 `400 INVALID_AI_PREVIEW_ROUTE` 다.
- 시간·체류시간 값이 올바르지 않으면 `400 INVALID_AI_PREVIEW_PLACE_SCHEDULE` 이다.
- 버전 충돌은 `409 AI_PREVIEW_MODIFICATION_CONFLICT` 다.

연동 API 엔드포인트

- `GET /api/ai/trip-previews/{previewId}`
- `PATCH /api/ai/trip-previews/{previewId}`

3.19.7 기능명: AI 일정 미리보기 확정 적용

기능 설명

- 사용자는 AI 미리보기를 실제 Trip으로 새로 생성하거나, 위시리스트 기반 미리보기인 경우 원본 Trip에 확정 적용한다.
- 일반 조건 및 저장 장소 기반 미리보기는 새 Trip을 생성한다.
- `TRIP_WISHLIST` 미리보기는 연결된 기존 Trip의 Day와 장소 구성을 갱신한다.

세부 로직 및 상태 변화

- Preview 소유권, 만료 여부와 검증 완료 상태를 확인한다.
- `GENERAL`, `SAVED_PLACES` Preview는 `Trip`, `TripDay`, `TripPlace` 를 하나의 트랜잭션으로 생성한다.
- `TRIP_WISHLIST` Preview는 연결된 Trip 소유권과 경로 수정 가능 여부를 다시 검증한 후 기존 Day·장소에 적용한다.
- 연결 Trip의 `cancelledAt` 이 존재하면 적용을 거부한다.
- 기존 Trip에 발행된 Post가 연결되어 있으면 AI 결과를 적용하지 않고 `409 TRIP_ROUTE_LOCKED_BY_PUBLISHED_POST` 를 반환한다.
- 동일 Preview를 중복 적용하면 기존 적용 결과 `tripId` 를 반환하도록 멱등 처리한다.

- Preview를 Trip에 적용해도 Preview의 TEMPORARY·PERMANENT 보관 상태는 별도 정책에 따른다.

입력 / Request DTO

- Path `previewId: Long`
- 별도 Body 없음

출력 / Response DTO

- `tripId: Long`
- `applyType: CREATED_NEW_TRIP | UPDATED_EXISTING_TRIP`
- `appliedAt: LocalDateTime`

예외사항 및 검증 로직

- 비로그인 요청은 `401 UNAUTHORIZED` 다.
- 존재하지 않는 Preview는 `404 AI_PREVIEW_NOT_FOUND` 다.
- 만료된 Preview는 `410 AI_PREVIEW_EXPIRED` 다.
- 타인 Preview는 `403 AI_PREVIEW_ACCESS_DENIED` 다.
- 기존 Trip 경로 잠금은 `409 TRIP_ROUTE_LOCKED_BY_PUBLISHED_POST` 다.
- 취소된 기존 Trip은 `409 TRIP_CANCELLED_READ_ONLY` 다.

연동 API 엔드포인트

- `POST /api/ai/trip-previews/{previewId}/apply`